TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN ĐIỆN TỬ - THÔNG TIN



ĐỒ ÁN MÔN HỌC

PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG QUẢN LÝ VÀ GIÁM SÁT NÔNG TRẠI DÂU TÍCH HỢP IOT VÀ AI

Sinh viên thực hiện: NGUYỄN TUẤN KIỆT
MSSV: 23139023
HOÀNG GIA PHÁT
MSSV: 23139033
ĐỖ THANH SƠN
MSSV: 23139039
NGUYỄN ĐÌNH THI
MSSV: 23139044

Giảng viên hướng dẫn: THS. HUỲNH HOÀNG HÀ

TP. HỒ CHÍ MINH, Ngày 27 tháng 10 năm 2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN ĐIỆN TỬ - THÔNG TIN



ĐỒ ÁN MÔN HỌC

PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG QUẢN LÝ VÀ GIÁM SÁT NÔNG TRẠI DÂU TÍCH HỢP IOT VÀ AI

Sinh viên thực hiện: NGUYỄN TUẤN KIỆT
MSSV: 23139023
HOÀNG GIA PHÁT
MSSV: 23139033
ĐỖ THANH SƠN
MSSV: 23139039
NGUYỄN ĐÌNH THI
MSSV: 23139044

Giảng viên hướng dẫn: THS. HUỲNH HOÀNG HÀ

TP. HỒ CHÍ MINH, Ngày 27 tháng 10 năm 2026

Mục lục

DANH MỤC BẢNG . . . . . **iv**

DANH MỤC HÌNH . . . . . **v**

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT . . . . . **vi**

LỜI CẢM ƠN . . . . . **vii**

LỜI CAM ĐOAN . . . . . **viii**

1 TỔNG QUAN ĐỀ TÀI **1**

1.1 Lý do chọn đề tài 1

1.2 Mục tiêu của đề tài 2

1.3 Đối tượng và phạm vi nghiên cứu 3

1.3.1 Đối tượng nghiên cứu 3

1.3.2 Phạm vi nghiên cứu 3

1.4 Phương pháp nghiên cứu 4

1.4.1 Phương pháp nghiên cứu lý thuyết 4

1.4.2 Phương pháp thực nghiệm 4

1.4.3 Phương pháp kiểm thử và đánh giá 5

2 CƠ SỞ LÝ THUYẾT **6**

2.1 Tổng quan về IoT trong nông nghiệp 6

2.2 Cơ sở dữ liệu và truyền dữ liệu với Supabase 7

2.2.1 REST API 7

2.2.2 Supabase Realtime 9

2.3 Nền tảng phát triển ứng dụng di động React Native 10

2.4 Cơ sở lý thuyết về học sâu trong nhận diện bệnh cây trồng 11

2.4.1 Mạng nơ-ron tích chập CNN 12

2.4.2 Học chuyển giao 13

2.5 Kiến trúc MobileNetV3-Large 14

2.5.1 Tích chập tách biệt theo chiều sâu 14

2.5.2 Khối Inverted Residual và Squeeze-and-Excitation 14

2.5.3 Hàm kích hoạt h-swish và cấu trúc MobileNetV3-Large 15

2.5.4 Triển khai suy luận trên thiết bị di động 16

3 THIẾT KẾ HỆ THỐNG 18
3.1 Sơ đồ tổng quan hệ thống 18
3.2 Thiết kế phần mềm nhúng 19
3.2.1 Cấu trúc chương trình ESP32 19
3.2.2 Phân công chân kết nối và chu kỳ xử lý 19
3.2.3 Đọc và hiệu chuẩn cảm biến 20
3.2.4 Thiết kế chế độ điều khiển 21
3.2.5 Thiết kế hệ thống cảnh báo 21
3.2.6 Khả năng phục hồi và cấu hình thiết bị 22
3.3 Thiết kế cơ sở dữ liệu và backend 22
3.3.1 Vai trò của Supabase và Cloudinary 22
3.3.2 Các bảng dữ liệu được sử dụng 22
3.3.3 Trao đổi dữ liệu Realtime 23
3.3.4 Ràng buộc giữa thiết bị và ứng dụng 23
3.4 Thiết kế mô hình học sâu 24
3.4.1 Tổ chức dữ liệu và các lớp phân loại 24
3.4.2 Tiền xử lý và tăng cường dữ liệu 24
3.4.3 Kiến trúc và chiến lược huấn luyện 25
3.4.4 Định dạng triển khai và giao diện dữ liệu cho ứng dụng 26
3.5 Thiết kế giao diện ứng dụng (UI/UX) 26
3.5.1 Cấu trúc điều hướng 26
3.5.2 Màn hình Trang chủ 27
3.5.3 Màn hình Điều khiển 28
3.5.4 Màn hình Cây trồng 28
3.5.5 Thiết kế màn hình nhận diện bệnh 28
3.6 Thiết kế luồng hoạt động 29
3.6.1 Luồng giám sát môi trường 29
3.6.2 Luồng điều khiển thủ công 29
3.6.3 Luồng điều khiển tự động 30
3.6.4 Luồng quản lý cây và ảnh 30
3.6.5 Luồng suy luận AI 30
3.7 Thiết kế phần cứng 30
3.7.1 Cấu trúc phần cứng 30
3.7.2 Nguyên tắc lắp đặt và vận hành 31
3.7.3 Phạm vi phần cứng hiện tại và khả năng mở rộng 31

4 KẾT QUẢ 33
4.1 Kết quả huấn luyện mô hình học sâu 33
4.1.1 Kết quả chuẩn bị dữ liệu 33
4.1.2 Kết quả xây dựng và huấn luyện mô hình 34

4.1.3 Đánh giá mô hình trên tập kiểm thử 35
4.1.4 Kết quả tích hợp mô hình TensorFlow Lite 36
4.2 Kết quả giao diện ứng dụng di động 39
4.2.1 Màn hình Trang chủ 39
4.2.2 Màn hình Điều khiển 41
4.2.3 Màn hình Cây trồng 43
4.3 Kết quả phần cứng và chương trình nhúng 45
4.3.1 Kết nối phần cứng 45
4.3.2 Thu thập và xử lý dữ liệu cảm biến 46
4.3.3 Kết quả điều khiển thủ công và tự động 46
4.3.4 Kết quả cơ chế cảnh báo và phục hồi 46
4.4 Kết quả tích hợp hệ thống 47
4.5 Đánh giá và nhận xét 49

5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 51
5.1 Kết luận 51
5.2 Hạn chế của đề tài 52
5.3 Hướng phát triển 53

MÃ NGUỒN ĐỀ TÀI 56

DANH MỤC BẢNG

3.1 Các chu kỳ vận hành chính của ESP32 21
3.2 Ngưỡng cảnh báo môi trường trong firmware 22
3.3 Phân bố dữ liệu sau khi kiểm tra và loại ảnh trùng 25

4.1 Phân bố dữ liệu sử dụng cho quá trình huấn luyện và đánh giá 34
4.2 Phân công chân phần cứng theo firmware hiện tại 45
4.3 Một số ngưỡng cảnh báo môi trường trong firmware 47
4.4 Mức độ hiện thực các chức năng trong phiên bản mã nguồn hiện tại 49

DANH MỤC HÌNH

1.1 Kiến trúc tổng thể của hệ thống 2

2.1 Kiến trúc phân lớp và luồng thông tin của hệ thống IoT. 6
2.2 Trao đổi dữ liệu cảm biến và lệnh điều khiển giữa ESP32 và Supabase qua REST API. 8
2.3 Cơ chế cập nhật dữ liệu giám sát thông qua Supabase Realtime. 9
2.4 Kiến trúc chức năng của ứng dụng React Native trong hệ thống. 11
2.5 Quá trình trích xuất đặc trưng và phân loại ảnh bằng mạng CNN. 12
2.6 Quy trình áp dụng học chuyển giao trong phân loại bệnh dâu tây. 13
2.7 Cấu trúc khối MobileNetV3 có tích hợp cơ chế Squeeze-and-Excitation. . . 15
2.8 Cấu trúc MobileNetV3-Large và đầu phân loại được điều chỉnh cho tám lớp của đề tài. 16
2.9 Luồng suy luận nhận diện bệnh dâu tây trực tiếp trên thiết bị di động. 17

3.1 Sơ đồ tổng quan hệ thống 18
3.2 Sơ đồ tổng quan hệ thống 20
3.3 Các bảng dữ liệu được sử dụng 23
3.4 Quy trình chuẩn bị dữ liệu, huấn luyện và xuất mô hình 26
3.5 Cấu trúc chức năng của ứng dụng React Native 27
3.6 Luồng vận hành tổng thể của hệ thống 29
3.7 Sơ đồ khối kết nối phần cứng theo firmware 31

4.1 Biểu đồ quá trình huấn luyện và fine-tuning mô hình 35
4.2 Kết quả đánh giá mô hình MobileNetV3-Large trên tập test 36
4.3 Ma trận nhầm lẫn của mô hình trên tập kiểm thử 36
4.4 Giao diện nhận diện bệnh bằng mô hình TensorFlow Lite trên điện thoại . . 38
4.5 Giao diện Trang chủ hiển thị dữ liệu cảm biến và biểu đồ môi trường 40
4.6 Giao diện điều khiển thiết bị và quản lý lịch hoạt động 42
4.7 Giao diện quản lý hồ sơ cây và hình ảnh cây trồng 44
4.8 Mô hình phần cứng của hệ thống giám sát và điều khiển 45
4.9 Kết quả tích hợp các thành phần của hệ thống 48

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Ký hiệu	Ý nghĩa
IoT	Internet of Things
AI	Artificial Intelligence
ESP32	Vi điều khiển ESP32
MQTT	Message Queuing Telemetry Transport
API	Application Programming Interface

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án môn học với đề tài "Phát triển ứng dụng di động quản lý và giám sát nông trại dâu tích hợp IoT và AI", nhóm chúng em đã nhận được rất nhiều sự hướng dẫn, hỗ trợ và động viên. Nhóm chúng em xin gửi lời cảm ơn chân thành đến ThS. Huỳnh Hoàng Hà, giảng viên hướng dẫn, đã tận tình hướng dẫn, đóng góp ý kiến và hỗ trợ nhóm trong suốt quá trình thực hiện đồ án.

Mặc dù nhóm đã cố gắng hoàn thiện đồ án với tinh thần nghiêm túc, do thời gian và kiến thức còn hạn chế nên đồ án khó tránh khỏi những thiếu sót. Nhóm chúng em rất mong nhận được những ý kiến đóng góp của quý thầy cô để có thể hoàn thiện đề tài tốt hơn.

Nhóm chúng em xin chân thành cảm ơn!

TP. Hồ Chí Minh, Ngày 27 tháng 10 năm 2026

Nhóm sinh viên thực hiện

LỜI CAM ĐOAN

Nhóm chúng em xin cam đoan rằng đồ án với đề tài "Phát triển ứng dụng di động quản lý và giám sát nông trại dâu tích hợp IoT và AI" là kết quả nghiên cứu và thực hiện của nhóm dưới sự hướng dẫn của ThS. Huỳnh Hoàng Hà.

Các nội dung, kết quả và số liệu được trình bày trong đồ án được nhóm thực hiện, tổng hợp và trình bày một cách trung thực. Những tài liệu, hình ảnh, dữ liệu và kết quả tham khảo từ các nguồn khác đều được ghi rõ nguồn trong tài liệu.

TP. Hồ Chí Minh, Ngày 27 tháng 10 năm 2026

Nhóm sinh viên thực hiện

CHƯƠNG 1

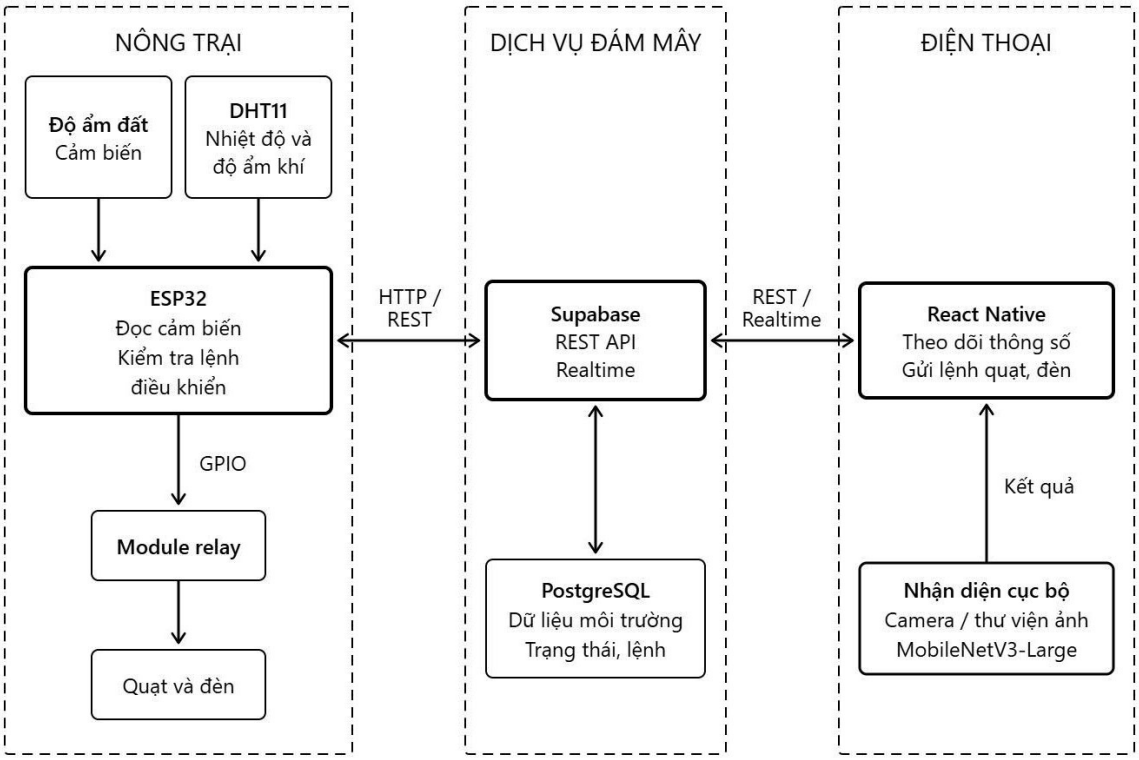
TỔNG QUAN ĐỀ TÀI

1.1 Lý do chọn đề tài

Dâu tây là một trong những loại cây trồng có giá trị kinh tế cao, đặc biệt phù hợp với các mô hình sản xuất nông nghiệp công nghệ cao. Tuy nhiên, cây dâu tây khá nhạy cảm với điều kiện môi trường và dễ bị ảnh hưởng bởi nhiều loại bệnh trên lá, hoa và quả như đốm lá góc cạnh, thán thư, mốc xám và bệnh phấn trắng. Khi không được phát hiện và xử lý kịp thời, bệnh có thể lây lan sang các cây khác, làm ảnh hưởng đến quá trình sinh trưởng, năng suất và chất lượng quả. Trong phương thức canh tác truyền thống, việc phát hiện bệnh trên cây dâu tây chủ yếu dựa vào quá trình quan sát trực tiếp của người trồng. Phương pháp này phụ thuộc nhiều vào kinh nghiệm của người canh tác và khả năng nhận biết các biểu hiện bất thường trên cây. Đối với những giai đoạn bệnh mới xuất hiện, các triệu chứng thường chưa rõ ràng nên việc nhận biết bằng mắt thường có thể gặp nhiều khó khăn. Bên cạnh đó, khi diện tích canh tác tăng lên, việc kiểm tra thủ công từng cây cũng đòi hỏi nhiều thời gian và công sức. Không chỉ vấn đề bệnh cây, các yếu tố môi trường như nhiệt độ, độ ẩm không khí và độ ẩm đất cũng có ảnh hưởng đáng kể đến quá trình sinh trưởng của cây dâu tây. Trong thực tế, các thông số này thường được kiểm tra bằng phương pháp thủ công hoặc chỉ được theo dõi tại một số thời điểm nhất định. Điều này khiến người trồng khó nắm bắt được sự thay đổi của môi trường trong suốt quá trình canh tác và hạn chế khả năng theo dõi dữ liệu trong thời gian dài.

Sự phát triển của Internet vạn vật và trí tuệ nhân tạo đã tạo điều kiện cho việc ứng dụng công nghệ vào lĩnh vực nông nghiệp. Các cảm biến có thể được sử dụng để thu thập dữ liệu môi trường liên tục, trong khi các mô hình học sâu có khả năng phân tích hình ảnh và hỗ trợ nhận diện bệnh trên cây trồng. Việc kết hợp hai hướng tiếp cận này có thể tạo ra một hệ thống vừa theo dõi được điều kiện canh tác, vừa hỗ trợ người trồng phát hiện sớm các dấu hiệu bất thường trên cây. Đặc biệt, các kiến trúc mạng nơ-ron tích chập có kích thước nhỏ như MobileNetV3 cho phép triển khai mô hình nhận diện trực tiếp trên các thiết bị có tài nguyên hạn chế. Với cách tiếp cận này, quá trình nhận diện bệnh có thể được thực hiện ngay trên điện thoại mà không cần gửi hình ảnh lên một máy chủ riêng để xử lý. Điều này giúp giảm sự phụ thuộc vào kết nối mạng, đồng thời rút ngắn thời gian phản hồi khi người dùng thực hiện nhận diện.

Xuất phát từ những vấn đề trên, nhóm lựa chọn thực hiện đề tài xây dựng hệ thống quản lý và giám sát nông trại dâu tây ứng dụng IoT và AI. Hệ thống tập trung vào việc thu thập dữ liệu môi trường thông qua ESP32, truyền và lưu trữ dữ liệu trên nền tảng Supabase, đồng thời cung cấp ứng dụng di động để người dùng theo dõi trạng thái nông trại và nhận diện bệnh trên lá, quả dâu tây bằng mô hình MobileNetV3-Large. Kiến trúc tổng thể của hệ thống được trình bày trong



Hình 1.1: Kiến trúc tổng thể của hệ thống

1.2 Mục tiêu của đề tài

Đề tài được thực hiện với mục tiêu xây dựng một hệ thống có khả năng hỗ trợ người trồng dâu theo dõi điều kiện môi trường và nhận diện một số bệnh phổ biến trên cây. Các mục tiêu cụ thể bao gồm:

- Thiết kế và xây dựng hệ thống giám sát môi trường sử dụng vi điều khiển ESP32, kết hợp cảm biến DHT11 để đo nhiệt độ, độ ẩm không khí và cảm biến độ ẩm đất để theo dõi tình trạng đất. Hệ thống có khả năng điều khiển các thiết bị như quạt và đèn thông qua relay.
- Xây dựng cơ chế trao đổi dữ liệu giữa ESP32 và nền tảng Supabase thông qua giao thức HTTP. Dữ liệu môi trường và trạng thái thiết bị được lưu trữ trên cơ sở dữ liệu để phục vụ việc theo dõi và quản lý.
- Phát triển ứng dụng di động bằng React Native, cho phép người dùng theo dõi các thông số môi trường, quan sát trạng thái thiết bị và gửi lệnh điều khiển đến hệ thống.
- Xây dựng và huấn luyện mô hình học sâu MobileNetV3-Large cho bài toán phân loại hình ảnh bệnh trên cây dâu tây. Mô hình được huấn luyện trên bộ dữ liệu gồm các nhóm bệnh phổ biến và lớp cây khỏe mạnh.

- Tích hợp mô hình nhận diện bệnh vào ứng dụng di động theo hướng xử lý trực tiếp trên thiết bị. Người dùng có thể sử dụng camera hoặc hình ảnh có sẵn để thực hiện nhận diện mà không cần phụ thuộc vào một máy chủ AI riêng.
- Kiểm thử và đánh giá hoạt động của toàn bộ hệ thống, bao gồm khả năng thu thập dữ liệu, truyền dữ liệu, điều khiển thiết bị, nhận diện bệnh và thời gian phản hồi của ứng dụng.

1.3 Đối tượng và phạm vi nghiên cứu

1.3.1 Đối tượng nghiên cứu

Đề tài tập trung nghiên cứu các đối tượng chính sau:

- Cây dâu tây và một số bệnh phổ biến xuất hiện trên lá, hoa và quả. Phạm vi phân loại gồm đốm lá góc cạnh, thán thư trên quả, cháy hoa, mốc xám, đốm lá, phấn trắng trên quả và phấn trắng trên lá.
- Vi điều khiển ESP32 và các cảm biến phục vụ việc thu thập thông số môi trường trong khu vực trồng dâu tây, bao gồm nhiệt độ, độ ẩm không khí và độ ẩm đất.
- Nền tảng Supabase được sử dụng để lưu trữ dữ liệu và hỗ trợ trao đổi thông tin giữa thiết bị phần cứng và ứng dụng di động.
- Kiến trúc mạng nơ-ron tích chập MobileNetV3-Large được nghiên cứu và áp dụng cho bài toán phân loại hình ảnh bệnh trên cây dâu tây.

1.3.2 Phạm vi nghiên cứu

Trong phạm vi của đồ án, hệ thống được giới hạn ở các nội dung sau:

- Thiết kế và xây dựng hệ thống giám sát môi trường sử dụng vi điều khiển ESP32, kết hợp cảm biến DHT11 để thu thập dữ liệu nhiệt độ và độ ẩm không khí, cảm biến độ ẩm đất LMS393 để theo dõi độ ẩm của đất. Hệ thống đồng thời có khả năng điều khiển quạt và đèn thông qua module relay, phục vụ việc giám sát và điều chỉnh các điều kiện môi trường trong khu vực trồng dâu tây
- Về dữ liệu hình ảnh, đề tài sử dụng bộ dữ liệu Strawberry Disease Detection Dataset làm nguồn dữ liệu chính và bổ sung thêm hình ảnh dâu tây khỏe mạnh từ bộ dữ liệu PlantVillage. Dữ liệu được tổ chức lại thành tám lớp, bao gồm bảy lớp bệnh và một lớp cây khỏe mạnh.
- Về mô hình AI, đề tài tập trung vào bài toán phân loại hình ảnh. Mô hình MobileNetV3-Large được huấn luyện bằng phương pháp học chuyển giao và sau đó được chuyển đổi để triển khai trên thiết bị di động.

- Về phần mềm, ứng dụng được phát triển bằng React Native và tập trung kiểm thử trên thiết bị Android.

1.4 Phương pháp nghiên cứu

Để hoàn thành các mục tiêu đề ra, nhóm kết hợp phương pháp nghiên cứu lý thuyết với phương pháp thực nghiệm và kiểm thử hệ thống.

1.4.1 Phương pháp nghiên cứu lý thuyết

Nhóm tiến hành tìm hiểu và tổng hợp các tài liệu liên quan đến Internet vạn vật trong nông nghiệp, vi điều khiển ESP32, cảm biến môi trường, giao thức HTTP, nền tảng Supabase và phương pháp phát triển ứng dụng di động bằng React Native. Bên cạnh đó, nhóm nghiên cứu các kiến thức cơ bản về học sâu, mạng nơ-ron tích chập và phương pháp học chuyển giao nhằm lựa chọn mô hình phù hợp cho bài toán nhận diện bệnh trên cây dâu tây.

Trong quá trình nghiên cứu mô hình AI, nhóm tập trung tìm hiểu kiến trúc MobileNetV3-Large do mô hình có cấu trúc tương đối gọn và phù hợp với định hướng triển khai trên thiết bị di động. Các đặc điểm chính của kiến trúc được trình bày cụ thể ở phần thiết kế và xây dựng mô hình trong các chương tiếp theo.

1.4.2 Phương pháp thực nghiệm

Phương pháp thực nghiệm được sử dụng xuyên suốt quá trình xây dựng hệ thống. Các công việc chính bao gồm:

- Thiết kế và lắp ráp mạch phần cứng sử dụng ESP32, cảm biến nhiệt độ, độ ẩm không khí, cảm biến độ ẩm đất và relay để điều khiển các thiết bị chấp hành.
- Xây dựng chương trình trên ESP32 để đọc dữ liệu cảm biến, gửi dữ liệu lên Supabase và nhận lệnh điều khiển từ ứng dụng.
- Thiết lập cơ sở dữ liệu và cơ chế trao đổi dữ liệu giữa ESP32 với ứng dụng di động thông qua Supabase.
- Thu thập, làm sạch và tổ chức dữ liệu hình ảnh thành các lớp tương ứng với từng loại bệnh và lớp cây khỏe mạnh.
- Huấn luyện mô hình MobileNetV3-Large bằng phương pháp học chuyển giao, sau đó đánh giá mô hình trên tập dữ liệu kiểm tra.
- Tích hợp mô hình đã huấn luyện vào ứng dụng di động để thực hiện quá trình nhận diện trực tiếp trên thiết bị.
- Tiến hành thử nghiệm hệ thống trong điều kiện mô phỏng thực tế nhằm đánh giá khả năng hoạt động của từng thành phần và sự phối hợp giữa các thành phần.

1.4.3 Phương pháp kiểm thử và đánh giá

Sau khi hoàn thành từng thành phần, nhóm tiến hành kiểm thử độc lập trước khi thực hiện kiểm thử tích hợp. Đối với phần cứng, các cảm biến được kiểm tra khả năng thu thập dữ liệu và relay được kiểm tra khả năng đóng ngắt thiết bị. Đối với hệ thống phần mềm, nhóm kiểm tra quá trình truyền, lưu trữ và cập nhật dữ liệu giữa ESP32, Supabase và ứng dụng di động. Đối với mô hình AI, kết quả được đánh giá dựa trên các chỉ số phù hợp với bài toán phân loại ảnh, đồng thời kiểm tra khả năng nhận diện trên các hình ảnh thực tế. Ứng dụng di động cũng được kiểm tra về khả năng xử lý hình ảnh, thời gian phản hồi và tính ổn định trong quá trình sử dụng. Cuối cùng, các thành phần được kết hợp để thực hiện kiểm thử toàn hệ thống theo quy trình từ thu thập dữ liệu tại ESP32, lưu trữ trên Supabase, hiển thị trên ứng dụng đến quá trình nhận diện bệnh bằng mô hình AI. Kết quả kiểm thử là cơ sở để đánh giá mức độ đáp ứng của hệ thống so với các mục tiêu ban đầu.

CHƯƠNG 2

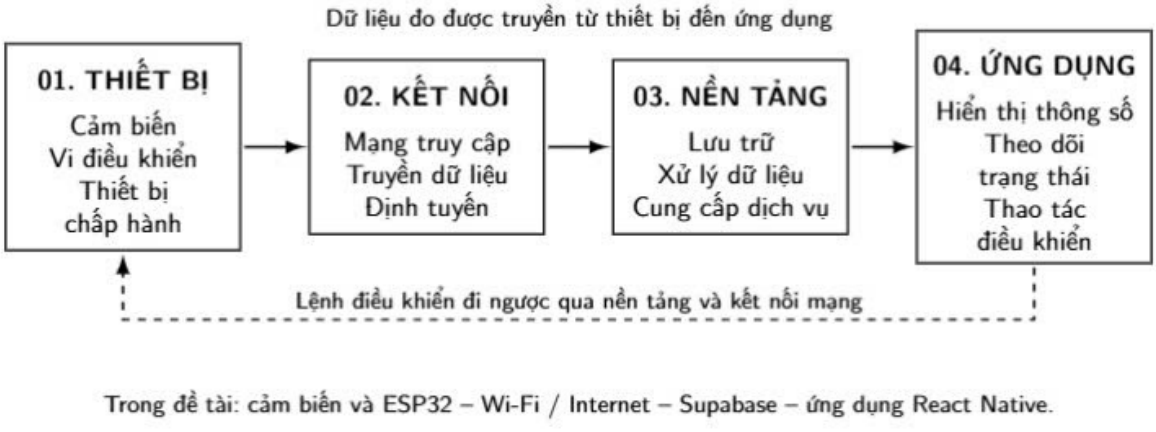
CƠ SỞ LÝ THUYẾT

Chương này trình bày những cơ sở lý thuyết và công nghệ nền tảng phục vụ việc xây dựng ứng dụng di động quản lý, giám sát nông trại dâu tây tích hợp Internet vạn vật (Internet of Things – IoT) và trí tuệ nhân tạo (Artificial Intelligence – AI). Nội dung tập trung vào mô hình IoT trong nông nghiệp, phương thức lưu trữ và trao đổi dữ liệu thông qua Supabase, nền tảng phát triển ứng dụng React Native, các nguyên lý của học sâu và học chuyển giao, cũng như kiến trúc MobileNetV3-Large phục vụ bài toán phân loại bệnh trên cây dâu tây. Các nội dung được trình bày theo thứ tự từ hệ thống thu thập dữ liệu, hạ tầng phần mềm đến mô hình nhận diện hình ảnh, qua đó làm cơ sở cho các quyết định thiết kế được đề cập ở chương tiếp theo.

2.1 Tổng quan về IoT trong nông nghiệp

Internet vạn vật là mô hình trong đó các đối tượng vật lý được trang bị cảm biến, bộ xử lý và khả năng kết nối mạng nhằm thu thập, trao đổi hoặc sử dụng dữ liệu từ môi trường. Khác với các hệ thống giám sát truyền thống chỉ hoạt động cục bộ, IoT tạo điều kiện để thông tin từ thiết bị được truy cập qua mạng, đồng thời cho phép người sử dụng gửi lệnh điều khiển từ xa. Những thành phần thường gặp trong hệ thống IoT bao gồm cảm biến, vi điều khiển, thiết bị chấp hành, hạ tầng truyền thông, nền tảng xử lý dữ liệu và ứng dụng phía người dùng.

Về nguyên lý, luồng thông tin của hệ thống được hình thành từ quá trình đo lường, truyền dữ liệu, lưu trữ, hiển thị và phản hồi điều khiển. Cảm biến chuyển các đại lượng vật lý như nhiệt độ hoặc độ ẩm thành tín hiệu điện; vi điều khiển đọc và xử lý sơ bộ các tín hiệu này, sau đó truyền bản ghi lên một dịch vụ trung gian. Ứng dụng khai thác dữ liệu để cung cấp thông tin cho người quản lý. Khi có yêu cầu điều khiển, lệnh được truyền theo chiều ngược lại để thiết bị chấp hành thực hiện hành động tương ứng.



Hình 2.1: Kiến trúc phân lớp và luồng thông tin của hệ thống IoT.

Hình 2.1 mô tả mối liên hệ giữa các thiết bị tại hiện trường, mạng truyền thông, nền tảng dữ liệu và ứng dụng giám sát. Mô hình này không chỉ hỗ trợ quan sát trạng thái môi trường mà còn tạo thành một vòng phản hồi giữa dữ liệu đo được và hoạt động điều khiển.

Trong nông nghiệp, việc giám sát liên tục có ý nghĩa vì điều kiện môi trường thay đổi theo thời gian và có thể ảnh hưởng đến sinh trưởng của cây trồng. Các thông số thường được theo dõi gồm nhiệt độ không khí, độ ẩm không khí, độ ẩm đất và cường độ ánh sáng. Nghiên cứu tổng quan của Ayaz và cộng sự cho thấy IoT có tiềm năng hỗ trợ các hoạt động theo dõi cây trồng, quản lý tài nguyên và tự động hóa trong sản xuất nông nghiệp, đồng thời đặt ra các thách thức về độ tin cậy truyền thông, năng lượng và khả năng triển khai thực tế [1].

Đối với mô hình trồng dâu tây của đề tài, ESP32 đảm nhiệm việc đọc dữ liệu từ cảm biến và giao tiếp với nền tảng dữ liệu thông qua Wi-Fi. Cảm biến DHT11 được dùng để đo nhiệt độ và độ ẩm không khí, còn cảm biến độ ẩm đất cung cấp thông tin về điều kiện tại khu vực rễ. Hệ thống sử dụng quạt và đèn làm các thiết bị chấp hành. Người dùng có thể quan sát số liệu trên điện thoại, thay đổi trạng thái thiết bị thủ công hoặc sử dụng cơ chế điều khiển theo ngưỡng đã thiết lập trong chương trình nhúng.

Cần phân biệt giữa chức năng *giám sát* và *điều khiển*. Chức năng giám sát tạo ra dữ liệu và cảnh báo để hỗ trợ ra quyết định; chức năng điều khiển tác động trở lại môi trường thông qua thiết bị chấp hành. Khi triển khai thực tế, các giá trị ngưỡng cần được lựa chọn theo điều kiện canh tác, còn dữ liệu cảm biến phải được kiểm tra tính hợp lệ để tránh thực hiện hành động dựa trên số đo sai lệch. Đây là những yêu cầu quan trọng khi chuyển từ mô hình thử nghiệm sang vận hành ổn định.

2.2 Cơ sở dữ liệu và truyền dữ liệu với Supabase

Supabase là nền tảng cung cấp các dịch vụ phía máy chủ (backend), sử dụng PostgreSQL làm hệ quản trị cơ sở dữ liệu cốt lõi. Bên cạnh việc lưu trữ thông tin dưới dạng bảng quan hệ, nền tảng còn cung cấp giao diện lập trình ứng dụng (API), cơ chế xác thực và dịch vụ truyền sự kiện thay đổi dữ liệu đến các máy khách. Các thành phần này giúp ứng dụng và thiết bị IoT khai thác cùng một nguồn dữ liệu mà không cần tự xây dựng toàn bộ hạ tầng máy chủ từ đầu [2].

Trong kiến trúc của đề tài, Supabase đóng vai trò trung gian giữa ESP32 và ứng dụng React Native. Các bản ghi cảm biến, trạng thái thiết bị và lịch sử điều khiển được lưu trong cơ sở dữ liệu. Nhờ có lớp trung gian này, ESP32 và điện thoại không phải duy trì một kết nối trực tiếp với nhau hoặc nằm trong cùng mạng Wi-Fi. Tuy nhiên, chất lượng đồng bộ vẫn phụ thuộc vào kết nối Internet và khả năng hoạt động của dịch vụ backend.

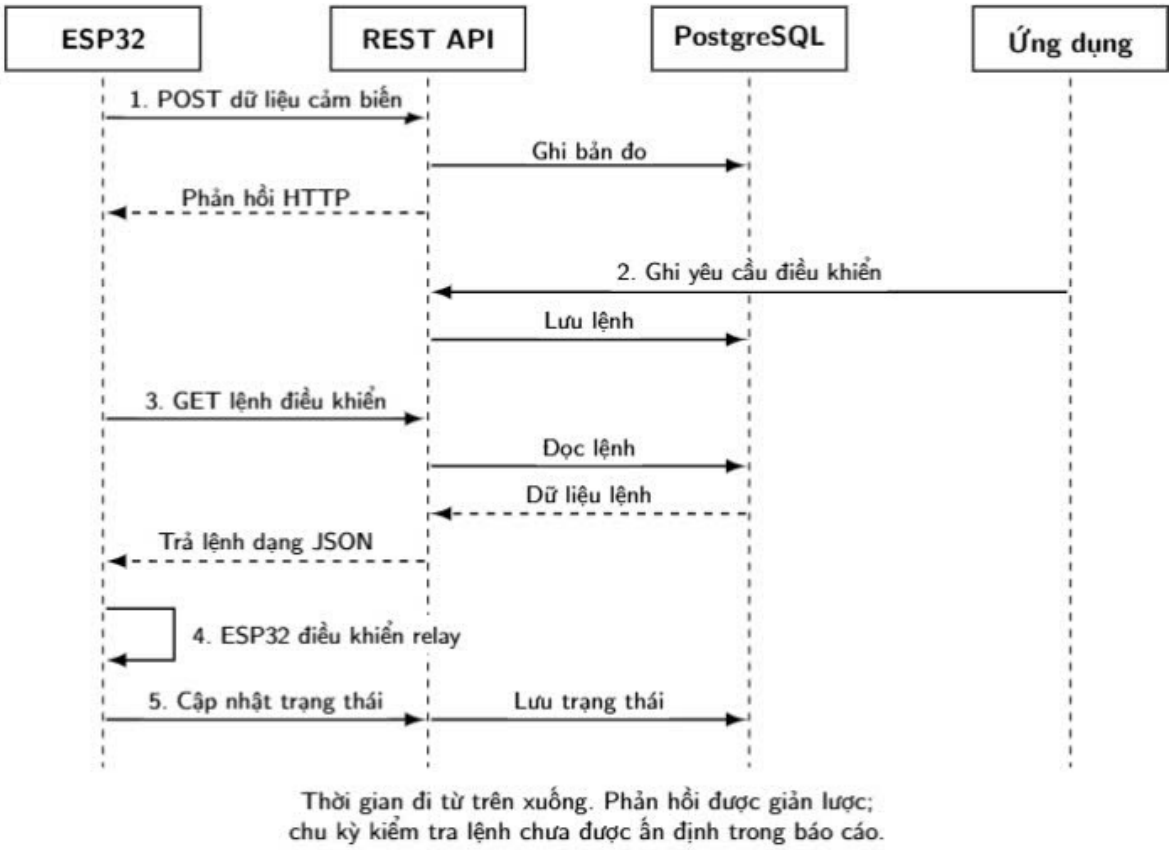
2.2.1 REST API

REST (Representational State Transfer) là một phong cách thiết kế giao diện giao tiếp cho các hệ thống phân tán. Trong thực tế, REST API thường sử dụng giao thức HTTP để máy khách gửi yêu cầu đến máy chủ nhằm đọc, tạo, cập nhật hoặc xóa tài nguyên. Một số phương

thức HTTP cơ bản gồm GET để truy vấn dữ liệu, POST để tạo bản ghi, PATCH để cập nhật một phần thông tin và DELETE để xóa tài nguyên.

Supabase cung cấp Data REST API dựa trên cấu trúc các bảng PostgreSQL. Khi gửi yêu cầu, thiết bị cần chỉ rõ địa chỉ API, loại thao tác, dữ liệu cần trao đổi và thông tin xác thực phù hợp. Dữ liệu thường được biểu diễn bằng định dạng JSON, thuận tiện cho việc xử lý trên vi điều khiển và ứng dụng di động. Đối với ESP32, cách giao tiếp theo mô hình yêu cầu–phản hồi giúp chương trình chủ động quyết định thời điểm gửi bản đo hoặc kiểm tra trạng thái điều khiển.

Trong đề tài, ESP32 gửi dữ liệu nhiệt độ, độ ẩm không khí và độ ẩm đất lên Supabase qua kết nối Wi-Fi. Khi nhận được yêu cầu điều khiển do ứng dụng ghi vào cơ sở dữ liệu, chương trình nhúng đọc trạng thái tương ứng và cập nhật đầu ra điều khiển quạt hoặc đèn. Hai chiều trao đổi này được thể hiện trong Hình 2.2.



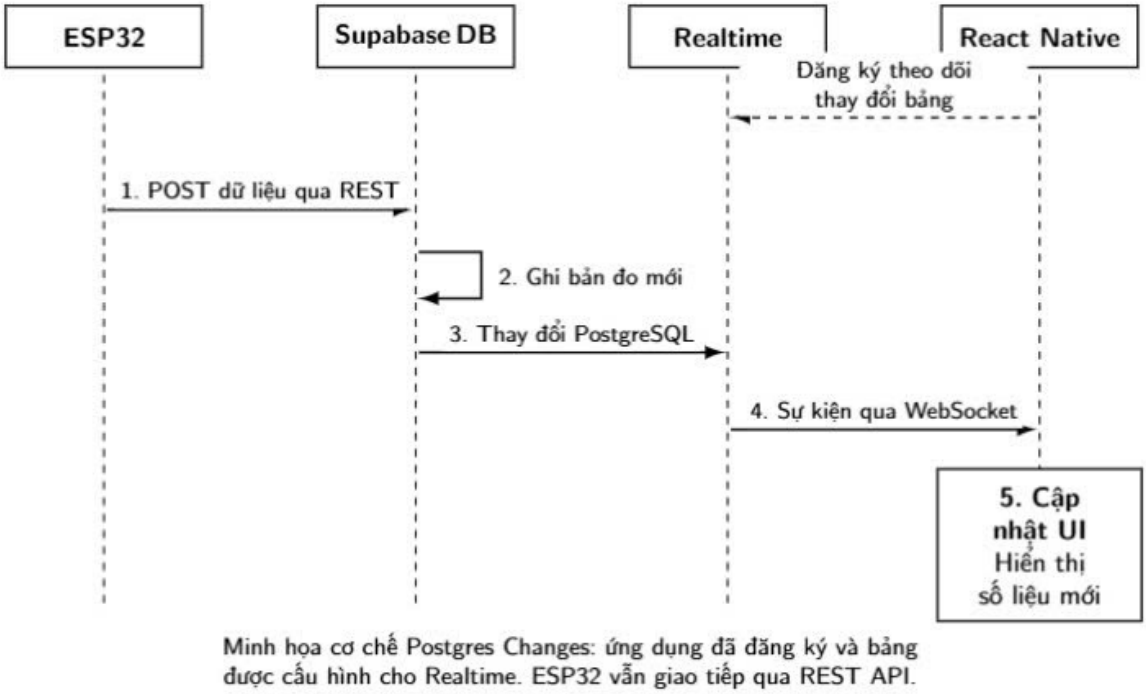
Hình 2.2: Trao đổi dữ liệu cảm biến và lệnh điều khiển giữa ESP32 và Supabase qua REST API.

REST API phù hợp với các tác vụ đọc và ghi theo chu kỳ, nhưng không tự động thông báo cho máy khách về mọi thay đổi của dữ liệu. Nếu chỉ sử dụng REST, ứng dụng phải thực hiện truy vấn lặp lại để phát hiện bản ghi mới. Vì vậy, đề tài kết hợp REST với Supabase Realtime nhằm đáp ứng hai nhu cầu giao tiếp khác nhau.

2.2.2 Supabase Realtime

Supabase Realtime cho phép một ứng dụng đang kết nối đăng ký nhận các sự kiện phát sinh từ cơ sở dữ liệu. Với cơ chế Postgres Changes, máy khách có thể theo dõi những thao tác như INSERT, UPDATE hoặc DELETE đối với bảng được cấu hình và nhận thông tin thay đổi thông qua kết nối WebSocket. Đây là cơ chế giao tiếp dựa trên sự kiện, trong đó ứng dụng không cần liên tục yêu cầu máy chủ trả về toàn bộ dữ liệu để kiểm tra trạng thái mới [2].

Đối với màn hình giám sát của đề tài, ứng dụng React Native trước tiên truy vấn các bản ghi cần thiết từ Supabase, sau đó đăng ký kênh Realtime để nhận các cập nhật tiếp theo. Khi ESP32 gửi một bản đo và cơ sở dữ liệu ghi nhận thành công, ứng dụng có thể nhận sự kiện và cập nhật giá trị hiển thị. Cách tổ chức này giúp giao diện phản ánh thay đổi của dữ liệu mà không phụ thuộc hoàn toàn vào việc tải lại thủ công.



Hình 2.3: Cơ chế cập nhật dữ liệu giám sát thông qua Supabase Realtime.

Hình 2.3 thể hiện vai trò trung gian của cơ sở dữ liệu trong việc phân phối sự kiện từ thiết bị đến ứng dụng. Cần lưu ý rằng Realtime không thay thế hoàn toàn REST: REST vẫn cần thiết cho truy vấn ban đầu, thao tác ghi dữ liệu và khôi phục trạng thái sau khi kết nối bị gián đoạn. Hệ thống cũng cần cấu hình quyền truy cập dữ liệu phù hợp để tránh cho một thiết bị hoặc người dùng truy cập ngoài phạm vi được phép.

Trong phạm vi đề tài, việc sử dụng Supabase REST API kết hợp Realtime giúp hạn chế số lượng thành phần phải triển khai, chẳng hạn không cần xây dựng một MQTT broker riêng. Tuy nhiên, đây là lựa chọn kiến trúc của hệ thống hiện tại, không có nghĩa REST và Realtime luôn thay thế được MQTT trong mọi ứng dụng IoT.

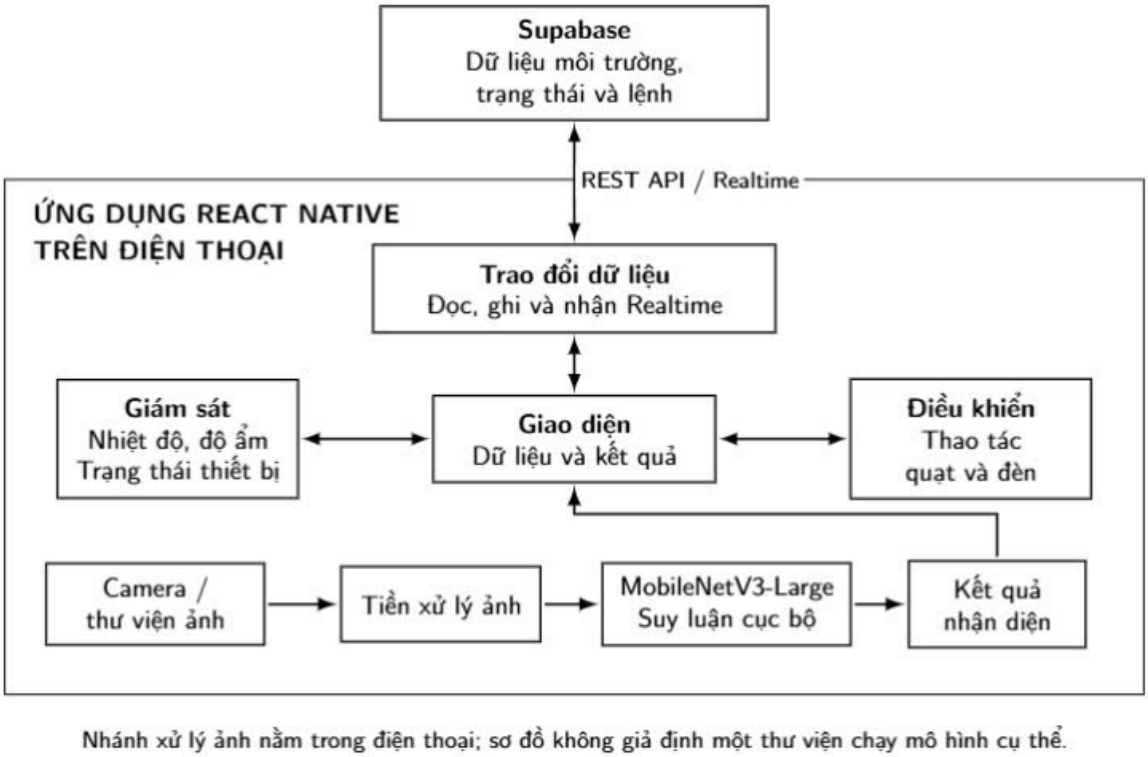
2.3 Nền tảng phát triển ứng dụng di động React Native

React Native là framework phát triển ứng dụng di động dựa trên React, cho phép xây dựng giao diện bằng JavaScript hoặc TypeScript và ánh xạ các thành phần giao diện sang các thành phần gốc của hệ điều hành. Thay vì phải phát triển riêng toàn bộ giao diện cho Android và iOS, lập trình viên có thể tái sử dụng phần lớn logic xử lý và cấu trúc giao diện giữa hai nền tảng, đồng thời tích hợp những chức năng đặc thù của thiết bị khi cần thiết [3].

Trong React Native, giao diện được xây dựng từ các *component*. Mỗi component đảm nhiệm một vai trò như hiển thị văn bản, hình ảnh, danh sách hoặc tiếp nhận thao tác người dùng. Trạng thái (state) và các cơ chế xử lý sự kiện cho phép nội dung giao diện thay đổi khi có dữ liệu mới. Phương thức tổ chức này phù hợp với ứng dụng giám sát, nơi các thông số môi trường và trạng thái thiết bị thường xuyên cần được cập nhật.

Một ưu điểm trong quá trình phát triển là Fast Refresh, cho phép quan sát nhanh nhiều thay đổi của mã giao diện mà không phải xây dựng lại toàn bộ ứng dụng ở mỗi lần chỉnh sửa. React Native cũng có các thư viện phục vụ kết nối mạng, vẽ biểu đồ, truy cập camera và lưu trữ dữ liệu. Tuy nhiên, việc tái sử dụng mã nguồn không đồng nghĩa mọi chức năng đều chạy giống nhau trên tất cả thiết bị; các mô-đun liên quan đến phần cứng vẫn cần được kiểm thử trên nền tảng triển khai cụ thể.

Đối với đề tài, React Native đảm nhiệm việc xây dựng giao diện người dùng trên điện thoại Android. Các nhóm chức năng chính bao gồm hiển thị nhiệt độ và độ ẩm, theo dõi lịch sử số đo, điều khiển quạt và đèn, quản lý thông tin cây trồng và tiếp nhận hình ảnh để phân loại bệnh. Ứng dụng sử dụng Supabase làm nguồn dữ liệu chung, kết hợp REST để truy vấn và Realtime để cập nhật các thay đổi phục vụ giám sát.



Hình 2.4: Kiến trúc chức năng của ứng dụng React Native trong hệ thống.

Hình 2.4 trình bày mối liên hệ giữa giao diện ứng dụng, dịch vụ dữ liệu và mô-đun AI. Trong đó, các chức năng giám sát và điều khiển chủ yếu trao đổi thông tin với backend, còn tác vụ nhận diện bệnh sử dụng ảnh được chọn từ camera hoặc thư viện của điện thoại. Việc phân tách hai nhóm chức năng giúp mô hình AI có thể suy luận trên thiết bị mà không bắt buộc gửi ảnh lên một máy chủ suy luận riêng.

2.4 Cơ sở lý thuyết về học sâu trong nhận diện bệnh cây trồng

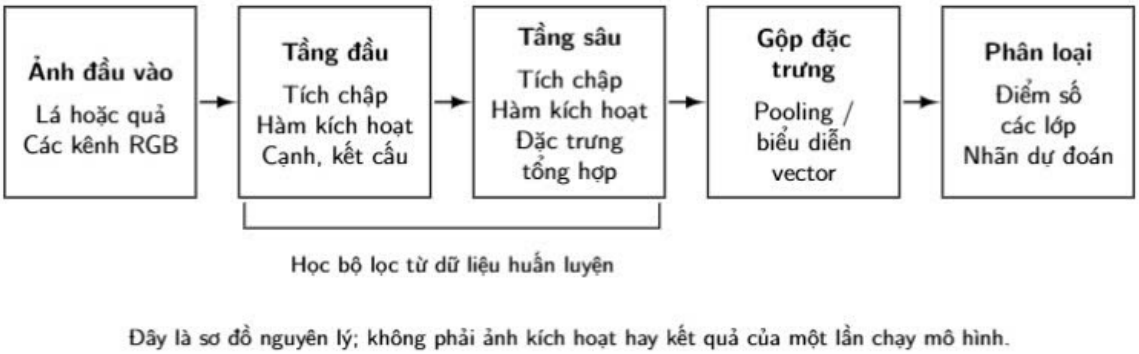
Học sâu (Deep Learning) là một nhánh của học máy, sử dụng các mạng nơ-ron nhiều lớp để xây dựng biểu diễn dữ liệu ở các mức độ trừu tượng khác nhau. Thông qua quá trình huấn luyện, các tham số của mô hình được điều chỉnh nhằm giảm sai lệch giữa kết quả dự đoán và nhãn mục tiêu. Khả năng tự học đặc trưng từ dữ liệu đã giúp học sâu trở thành một hướng tiếp cận quan trọng trong nhận dạng hình ảnh và nhiều bài toán xử lý tín hiệu [4].

Đối với bài toán nhận diện bệnh dâu tây, đầu vào là ảnh của lá, quả hoặc bộ phận cây trồng, còn đầu ra là lớp biểu thị tình trạng tương ứng. Những biểu hiện như sự thay đổi màu sắc, đốm bệnh, mốc hoặc biến dạng bề mặt có thể mang thông tin phục vụ phân loại. Tuy nhiên, các biểu hiện này thường chịu ảnh hưởng của ánh sáng, góc chụp, nền ảnh và giai đoạn phát triển của bệnh. Do đó, chất lượng dữ liệu và khả năng khái quát của mô hình giữ vai trò quan trọng bên cạnh kiến trúc mạng.

2.4.1 Mạng nơ-ron tích chập CNN

Mạng nơ-ron tích chập (Convolutional Neural Network – CNN) là một nhóm mô hình học sâu phù hợp với dữ liệu có cấu trúc không gian như hình ảnh. Thành phần cơ bản của CNN là lớp tích chập, trong đó các bộ lọc (kernel) được trượt trên ảnh hoặc bản đồ đặc trưng để phát hiện những mẫu cục bộ. Trong quá trình huấn luyện, trọng số của các bộ lọc được cập nhật để trích xuất những đặc trưng có ích cho nhiệm vụ phân loại.

Các lớp tích chập ở phần đầu mạng thường ghi nhận những thông tin tương đối đơn giản như cạnh, đường viền hoặc thay đổi độ sáng. Các lớp tiếp theo kết hợp những mẫu cục bộ để tạo ra biểu diễn phức tạp hơn, chẳng hạn vùng có kết cấu bất thường hoặc hình dạng đặc trưng của đối tượng. Sau các giai đoạn trích xuất đặc trưng, mạng sử dụng một đầu phân loại để tạo ra điểm dự đoán cho từng lớp.



Hình 2.5: Quá trình trích xuất đặc trưng và phân loại ảnh bằng mạng CNN.

Hình 2.5 mô tả quá trình chuyển ảnh đầu vào thành các bản đồ đặc trưng và kết quả phân loại. Đối với nhận diện bệnh cây trồng, CNN có khả năng học kết hợp nhiều tín hiệu trực quan thay vì phụ thuộc vào một dấu hiệu đơn lẻ do người thiết kế quy định. Mặc dù vậy, mô hình vẫn có thể dự đoán sai khi ảnh đầu vào khác biệt đáng kể so với dữ liệu đã được sử dụng trong huấn luyện.

Với bài toán gồm K lớp, đầu phân loại thường sử dụng hàm Softmax để chuyển các giá trị đầu ra z_k thành phân bố điểm số có tổng bằng một:

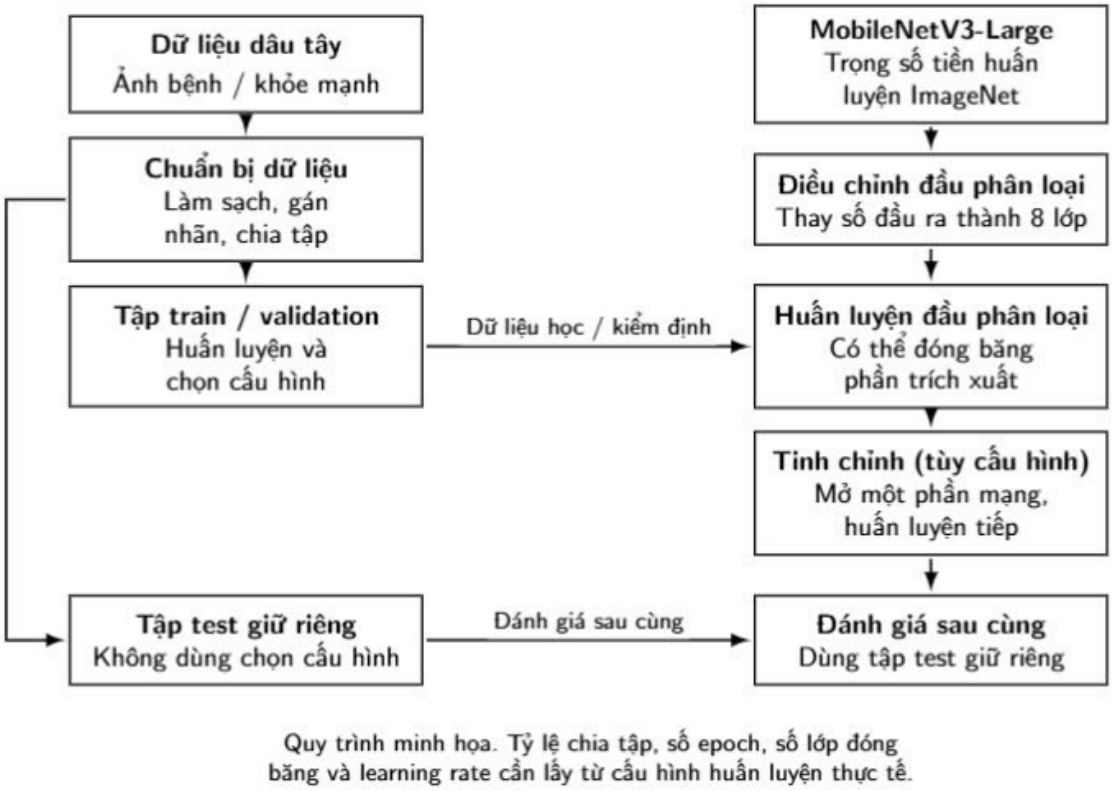
$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, 2, \ldots, K. \tag{2.1}$$

Trong đề tài, $K = 8$ tương ứng với tám nhóm bệnh hoặc trạng thái khỏe mạnh của dâu tây. Lớp được dự đoán là lớp có giá trị p_k lớn nhất. Cần lưu ý rằng điểm Softmax là đầu ra của mô hình, không mặc nhiên tương đương với xác suất đúng đã được hiệu chuẩn trên dữ liệu thực tế.

2.4.2 Học chuyển giao

Học chuyển giao (Transfer Learning) là phương pháp sử dụng tri thức đã được mô hình tiếp thu từ một miền dữ liệu hoặc một nhiệm vụ trước đó để hỗ trợ giải quyết nhiệm vụ mới. Thay vì huấn luyện toàn bộ mạng từ các trọng số khởi tạo ngẫu nhiên, phương pháp này tận dụng một mô hình đã được tiền huấn luyện, từ đó có thể giảm chi phí huấn luyện và cải thiện khả năng học khi bộ dữ liệu đích có quy mô hạn chế [5].

Đối với phân loại hình ảnh, một chiến lược phổ biến là sử dụng mạng đã được tiền huấn luyện trên ImageNet làm bộ trích xuất đặc trưng (backbone). Phần đầu phân loại gốc được thay bằng một lớp hoặc một nhóm lớp mới phù hợp với số lớp của bài toán đích. Trong giai đoạn đầu, backbone thường được đóng băng để huấn luyện phần phân loại mới. Sau đó, có thể mở một phần các lớp cuối của backbone và tiếp tục huấn luyện với tốc độ học nhỏ hơn nhằm điều chỉnh các đặc trưng theo ảnh của bài toán cụ thể.



Hình 2.6: Quy trình áp dụng học chuyển giao trong phân loại bệnh dâu tây.

Hình 2.6 minh họa việc kế thừa backbone tiền huấn luyện và thay thế đầu phân loại cho tập dữ liệu bệnh dâu tây. Trong đề tài, MobileNetV3-Large được sử dụng theo hướng học chuyển giao, với đầu ra được điều chỉnh cho tám lớp. Cách tiếp cận này tận dụng các đặc trưng hình ảnh đã học được, đồng thời cho phép mô hình thích nghi với dấu hiệu bệnh đặc trưng trong tập dữ liệu đang sử dụng.

Học chuyển giao không bảo đảm mô hình sẽ hoạt động tốt trong mọi tình huống. Nếu dữ liệu huấn luyện khác nhiều so với ảnh chụp thực tế, chẳng hạn về thiết bị camera, điều kiện ánh sáng hoặc mức độ che khuất, chất lượng dự đoán có thể suy giảm. Vì vậy, kết quả

huấn luyện cần được kiểm chứng trên tập dữ liệu độc lập và, khi triển khai, nên tiếp tục đánh giá bằng ảnh được thu thập trong môi trường sử dụng thực tế.

2.5 Kiến trúc MobileNetV3-Large

Các mô hình CNN có nhiều lớp và số lượng phép tính lớn có thể đạt khả năng biểu diễn mạnh, nhưng việc triển khai trên điện thoại thường bị giới hạn bởi tốc độ xử lý, bộ nhớ và mức tiêu thụ năng lượng. Họ MobileNet được phát triển nhằm giảm chi phí tính toán trong khi vẫn duy trì khả năng học đặc trưng hình ảnh. MobileNetV3 kế thừa các nguyên lý của các phiên bản trước, đồng thời kết hợp tối ưu kiến trúc dựa trên phần cứng với những cải tiến về khối tích chập và hàm kích hoạt [6].

MobileNetV3 có hai biến thể chính là MobileNetV3-Large và MobileNetV3-Small, được thiết kế cho những điều kiện tài nguyên khác nhau. Trong đề tài, MobileNetV3-Large được chọn làm backbone của mô hình phân loại bệnh dâu tây vì hướng đến sự cân bằng giữa khả năng trích xuất đặc trưng và yêu cầu triển khai trên thiết bị di động. Việc lựa chọn này phù hợp với mục tiêu thực hiện suy luận cục bộ trên điện thoại thay vì phụ thuộc vào một máy chủ xử lý ảnh.

2.5.1 Tích chập tách biệt theo chiều sâu

Trong tích chập thông thường, một bộ lọc tạo ra đặc trưng bằng cách xử lý đồng thời nhiều kênh đầu vào. Với tích chập tách biệt theo chiều sâu (Depthwise Separable Convolution), quá trình được chia thành hai bước. Trước hết, tích chập depthwise áp dụng bộ lọc không gian độc lập trên từng kênh. Sau đó, tích chập pointwise với kernel 1×1 kết hợp thông tin giữa các kênh để tạo ra số kênh đầu ra mong muốn.

Nếu kernel có kích thước $D_k \times D_k$, số kênh đầu vào là M và số kênh đầu ra là N, số phép nhân cơ bản cho mỗi vị trí không gian của tích chập thông thường tỷ lệ với $D_k^2 MN$. Đối với depthwise separable convolution, đại lượng tương ứng tỷ lệ với $D_k^2 M + MN$. Tỷ số chi phí tính toán xấp xỉ là:

$$\frac{D_k^2 M + MN}{D_k^2 MN} = \frac{1}{N} + \frac{1}{D_k^2}. \tag{2.2}$$

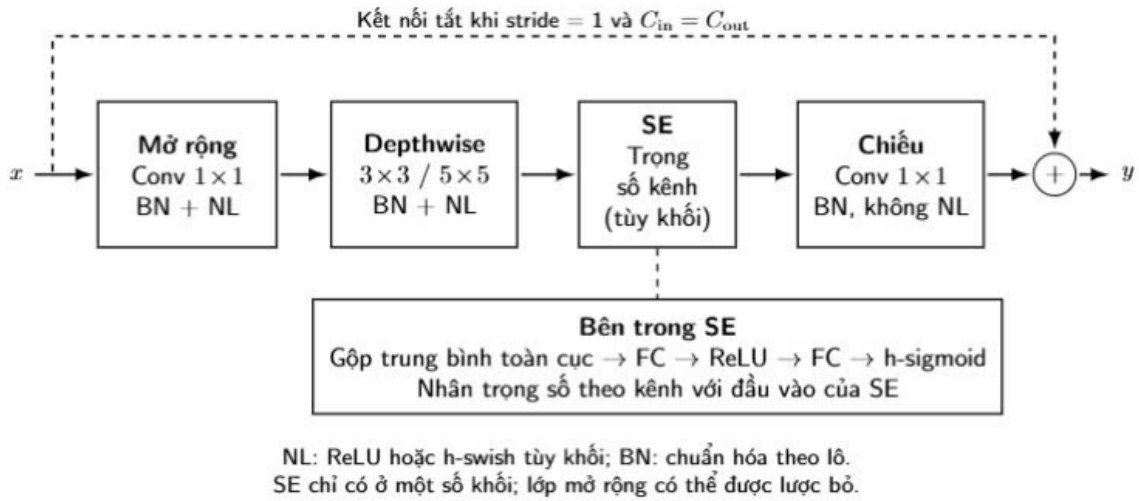
Biểu thức trên chỉ phản ánh chi phí phép nhân của hai dạng tích chập trong những điều kiện tương ứng; thời gian thực thi trên điện thoại còn phụ thuộc vào phần cứng, cách tổ chức bộ nhớ và thư viện suy luận. Dù vậy, đây là cơ sở giải thích vì sao các phép tích chập tách biệt thường được sử dụng trong những mạng CNN gọn nhẹ.

2.5.2 Khối Inverted Residual và Squeeze-and-Excitation

Trong MobileNetV3, một thành phần quan trọng là khối bottleneck theo nguyên lý *Inverted Residual*. Khối này thường bắt đầu bằng phép tích chập pointwise để mở rộng số kênh đặc trưng, tiếp theo là phép tích chập depthwise nhằm xử lý thông tin không gian, rồi sử

dụng một phép chiếu pointwise để đưa số kênh về kích thước đầu ra. Khi kích thước tensor và số kênh phù hợp, kết nối tắt (skip connection) có thể cộng trực tiếp đặc trưng đầu vào với đầu ra của khối.

Cơ chế Squeeze-and-Excitation (SE) được tích hợp ở một số khối nhằm tăng khả năng khai thác mối quan hệ giữa các kênh. Ở bước *squeeze*, thông tin không gian của mỗi kênh được tổng hợp thành một giá trị đại diện, thường bằng Global Average Pooling. Ở bước *excitation*, các giá trị này được đưa qua một mạng nhỏ để sinh ra trọng số cho từng kênh. Cuối cùng, các bản đồ đặc trưng được nhân với trọng số tương ứng, giúp mô hình điều chỉnh mức đóng góp của từng kênh theo nội dung ảnh.



Hình 2.7: Cấu trúc khối MobileNetV3 có tích hợp cơ chế Squeeze-and-Excitation.

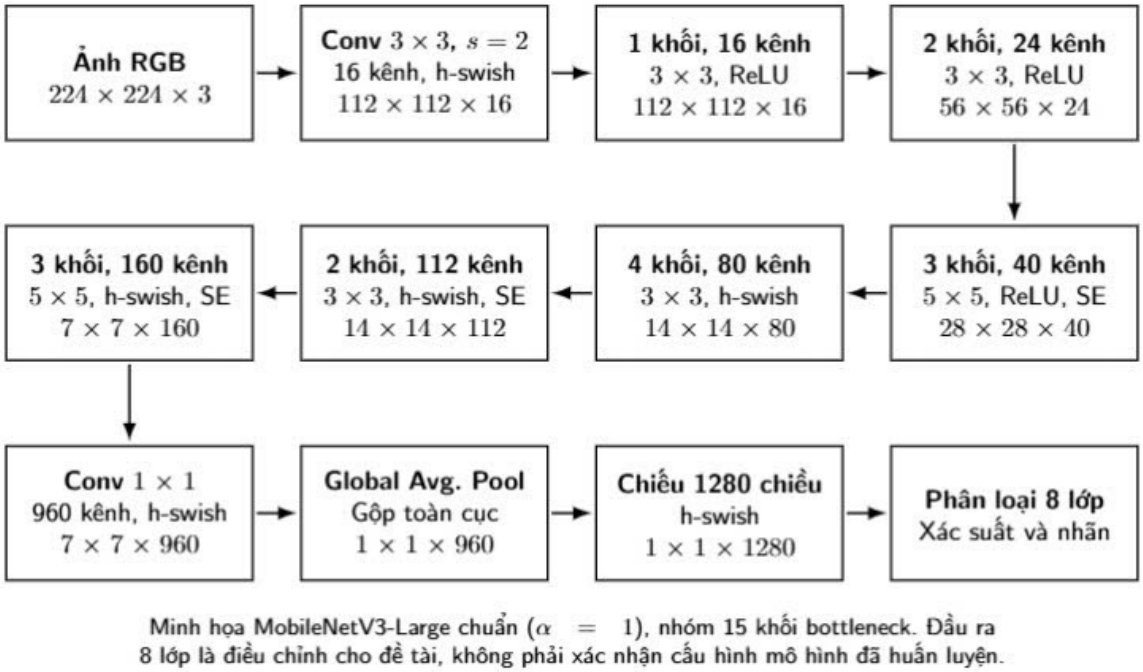
Hình 2.7 minh họa đường đi của dữ liệu qua các lớp mở rộng kênh, tích chập depthwise, cơ chế SE và lớp chiếu ở đầu ra. Sự kết hợp này hỗ trợ mô hình trích xuất đặc trưng hiệu quả mà không cần tăng số lượng phép tính theo cách của tích chập thông thường.

2.5.3 Hàm kích hoạt h-swish và cấu trúc MobileNetV3-Large

Bên cạnh cách tổ chức khối tích chập, MobileNetV3 sử dụng hàm kích hoạt h-swish ở một số vị trí trong mạng. Hàm này là một xấp xỉ có chi phí tính toán thuận tiện hơn của hàm swish, được định nghĩa như sau:

$$\text{h-swish}(x) = x\frac{\text{ReLU6}(x+3)}{6}, \tag{2.3}$$

trong đó $\text{ReLU6}(u) = \min(\max(u,0),6)$. Cách biểu diễn này sử dụng các phép toán đơn giản, góp phần hỗ trợ việc triển khai mạng trên phần cứng di động. Không phải tất cả lớp của MobileNetV3 đều sử dụng cùng một hàm kích hoạt; cấu hình thực tế còn bao gồm các lớp dùng ReLU.

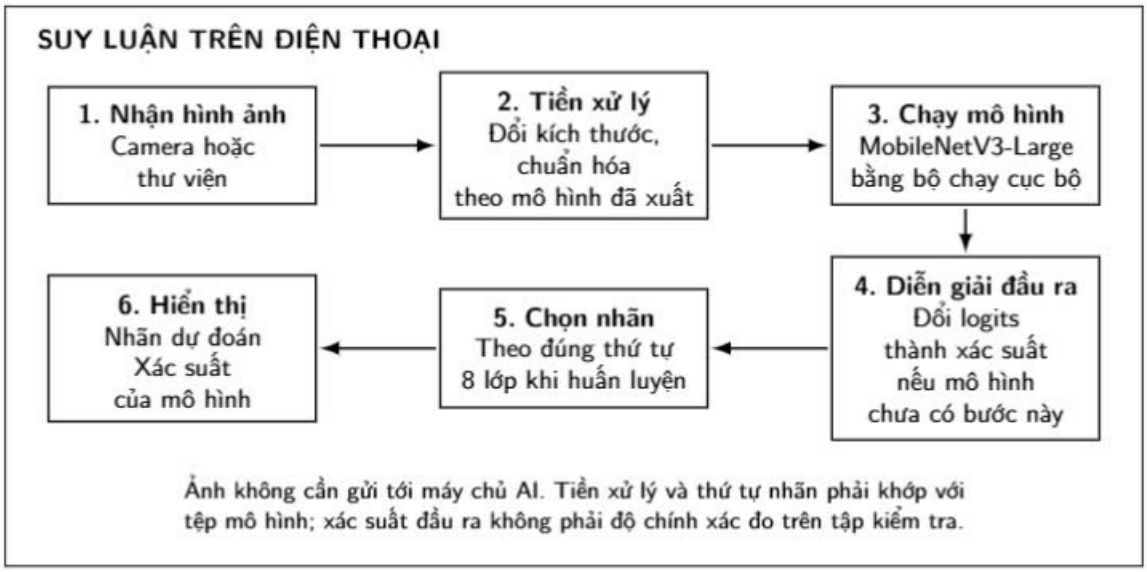


Hình 2.8: Cấu trúc MobileNetV3-Large và đầu phân loại được điều chỉnh cho tám lớp của đề tài.

Hình 2.8 trình bày các nhóm khối chính trong backbone và phần phân loại được điều chỉnh. Với bài toán của đề tài, ảnh sau tiền xử lý có kích thước $224 \times 224 \times 3$ được đưa vào mạng để trích xuất đặc trưng. Phần phân loại sử dụng đặc trưng từ backbone và tạo ra tám giá trị đầu ra, tương ứng với các lớp tình trạng cây trồng trong tập dữ liệu. Trong quy trình huấn luyện, việc sử dụng trọng số tiền huấn luyện kết hợp điều chỉnh đầu phân loại giúp tận dụng kiến trúc mạng đã được tối ưu cho xử lý ảnh.

2.5.4 Triển khai suy luận trên thiết bị di động

Sau khi huấn luyện, mô hình cần được chuyển sang định dạng tương thích với môi trường suy luận trên điện thoại. Trong đề tài, TensorFlow Lite (TFLite) được sử dụng để thực hiện bước này, cho phép ứng dụng tải mô hình và chạy suy luận cục bộ. Luồng xử lý bắt đầu từ việc lấy ảnh bằng camera hoặc thư viện, điều chỉnh kích thước và định dạng ảnh theo đầu vào của mô hình, thực hiện suy luận, sau đó chuyển kết quả dự đoán sang nhãn hiển thị cho người dùng.



Hình 2.9: Luồng suy luận nhận diện bệnh dâu tây trực tiếp trên thiết bị di động.

Hình 2.9 mô tả luồng suy luận từ ảnh đầu vào đến kết quả nhận diện. Để kết quả giữa môi trường huấn luyện và ứng dụng nhất quán, các bước tiền xử lý phải phù hợp với cấu hình mô hình đã xuất, bao gồm thứ tự kênh màu, kích thước tensor, kiểu dữ liệu và miền giá trị pixel. Đồng thời, thứ tự tám nhãn đầu ra phải được bảo toàn khi tích hợp vào ứng dụng.

Suy luận cục bộ giúp hạn chế việc truyền ảnh tới một dịch vụ AI từ xa và cho phép chức năng nhận diện tiếp tục hoạt động trong một số tình huống không có kết nối Internet, miễn là mô hình và các thành phần cần thiết đã có sẵn trên thiết bị. Tuy nhiên, khả năng vận hành thực tế còn phụ thuộc vào cấu hình điện thoại, thời gian suy luận, dung lượng mô hình và chất lượng ảnh chụp. Vì vậy, hiệu quả của hệ thống cần được đánh giá không chỉ bằng độ chính xác phân loại mà còn bằng khả năng đáp ứng trên thiết bị triển khai.

Như vậy, việc kết hợp IoT, Supabase và React Native hình thành nền tảng thu thập, lưu trữ, giám sát và điều khiển dữ liệu nông trại; trong khi học sâu, học chuyển giao và MobileNetV3-Large cung cấp cơ sở cho chức năng nhận diện bệnh bằng hình ảnh. Những nguyên lý trên là cơ sở để trình bày thiết kế phần cứng, phần mềm, mô hình AI và luồng hoạt động của hệ thống ở Chương 3.

CHƯƠNG 3

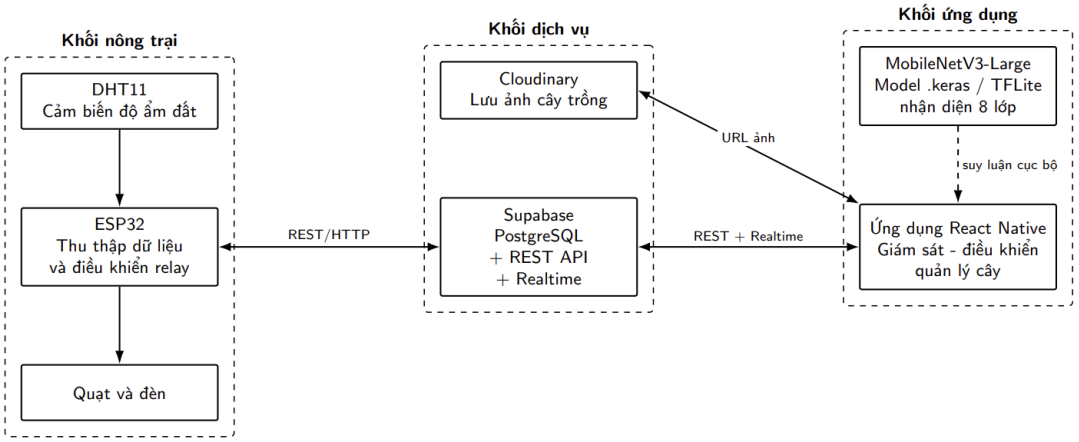
THIẾT KẾ HỆ THỐNG

Chương này trình bày thiết kế hệ thống dựa trên ba nhóm mã nguồn đã xây dựng cho đề tài: chương trình nhúng trên ESP32, mã huấn luyện mô hình MobileNetV3-Large và ứng dụng di động React Native. Nội dung tập trung mô tả cách các thành phần được tổ chức, cách dữ liệu được trao đổi, các tham số vận hành chính và điểm tích hợp giữa IoT, backend, ứng dụng và mô hình học sâu. Các thông số trong chương được lấy trực tiếp từ mã nguồn hiện tại; những chức năng chưa xuất hiện trong mã nguồn được ghi rõ là phần thiết kế mở rộng, tránh nhầm lẫn với chức năng đã hoàn thiện.

3.1 Sơ đồ tổng quan hệ thống

Hệ thống được tổ chức thành ba khối chính: khối nông trại, khối dịch vụ dữ liệu và khối ứng dụng di động. Khối nông trại sử dụng ESP32 để đọc cảm biến, thực hiện điều khiển relay và trao đổi dữ liệu với Supabase qua HTTP/REST. Khối dịch vụ sử dụng Supabase/PostgreSQL làm nơi lưu dữ liệu cảm biến, trạng thái thiết bị, cảnh báo, lịch sử điều khiển và hồ sơ cây trồng. Ứng dụng React Native đọc dữ liệu qua REST, đăng ký Realtime để cập nhật giao diện khi có thay đổi, đồng thời gửi lệnh điều khiển trở lại cơ sở dữ liệu. Ảnh cây trồng được tải lên Cloudinary, trong khi Supabase lưu URL ảnh và thông tin liên kết với từng cây.

Mô hình AI được huấn luyện độc lập bằng TensorFlow/Keras và được xuất sang định dạng TFLite để chạy suy luận cục bộ trên điện thoại. Mô-đun TFLite đã được tích hợp vào ứng dụng React Native, tiếp nhận ảnh từ camera hoặc thư viện, thực hiện tiền xử lý và trả về kết quả phân loại trong tám nhóm bệnh/khỏe mạnh. Quá trình suy luận diễn ra trên thiết bị, trong khi chức năng lưu trữ ảnh và đồng bộ dữ liệu vẫn sử dụng Cloudinary và Supabase.



Hình 3.1: Sơ đồ tổng quan hệ thống

3.2 Thiết kế phần mềm nhúng

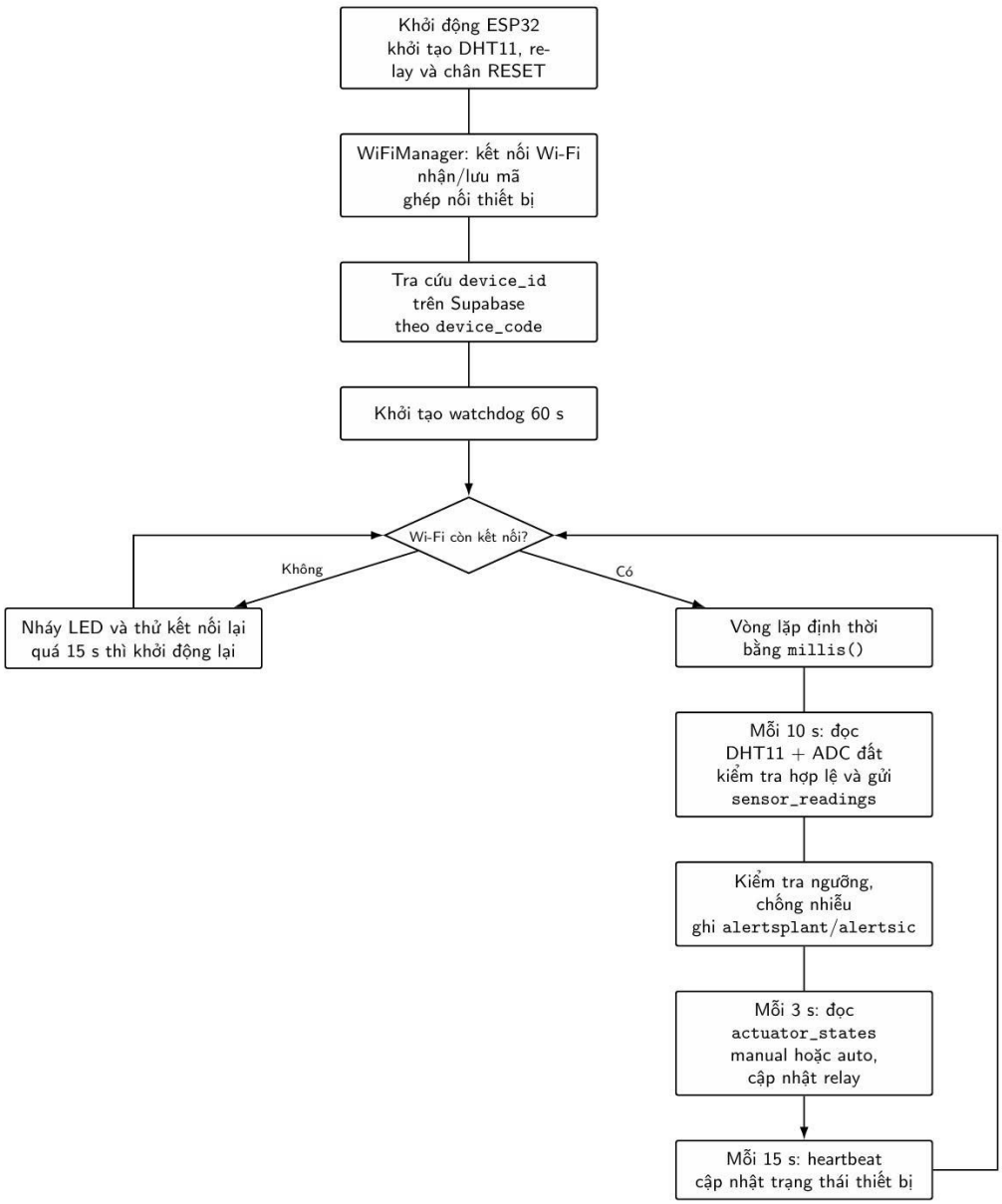
3.2.1 Cấu trúc chương trình ESP32

Firmware được viết cho ESP32 và sử dụng các thư viện WiFi, WiFiManager, HTTP-Client, ArduinoJson, Preferences, DHT và watchdog của ESP-IDF. Chương trình được chia thành các nhóm chức năng: cấu hình Wi-Fi và ghép nối thiết bị; đọc cảm biến; giao tiếp Supabase; điều khiển relay; xử lý cảnh báo; giám sát lỗi cảm biến; watchdog và reset thiết bị.

Khi khởi động, ESP32 khởi tạo DHT11, relay, nút reset và LED trạng thái. Tiếp theo, thiết bị sử dụng WiFiManager để kết nối mạng. Nếu chưa có mã ghép nối, ESP32 mở cổng cấu hình để người dùng nhập thông tin Wi-Fi và device_code. Mã này được lưu bằng Preferences để có thể tái sử dụng sau khi khởi động lại. Sau khi kết nối, firmware tra cứu bảng devices để lấy device_id tương ứng. Chỉ khi lấy được định danh hợp lệ, vòng lặp giám sát mới bắt đầu.

3.2.2 Phân công chân kết nối và chu kỳ xử lý

Bảng 3.1 tổng hợp các chân phần cứng được khai báo trực tiếp trong firmware.



Hình 3.2: Sơ đồ tổng quan hệ thống

Firmware không dùng delay() dài để tổ chức các tác vụ lặp mà dựa chủ yếu trên millis() và các mốc thời gian độc lập. Các chu kỳ chính được trình bày trong Bảng 3.2.

3.2.3 Đọc và hiệu chuẩn cảm biến

DHT11 cung cấp nhiệt độ và độ ẩm không khí. Firmware không chỉ kiểm tra giá trị NaN mà còn giới hạn miền hợp lệ của dữ liệu DHT11 trong khoảng 0–50°C đối với nhiệt độ và 20–90% đối với độ ẩm không khí. Cách kiểm tra này giúp loại các giá trị rác có thể xuất hiện khi cảm biến chập chờn nhưng vẫn trả về một số hữu hạn.

Đối với cảm biến độ ẩm đất, firmware hiệu chuẩn theo hai giá trị ADC đo thực tế: SOIL_RAW_DRY = 4095 ứng với trạng thái khô và SOIL_RAW_WET = 2110 ứng với trạng thái ướt. Giá trị analog được ánh xạ tuyến tính về miền 0–100% và giới hạn bằng hàm

Bảng 3.1: Các chu kỳ vận hành chính của ESP32

Tác vụ	Chu kỳ	Mục đích
Gửi dữ liệu cảm biến	10 s	Ghi nhiệt độ, độ ẩm không khí và độ ẩm đất vào sensor_readings
Đọc trạng thái điều khiển	3 s	Đọc actuator_states để xác định chế độ manual/auto và trạng thái relay
Heartbeat thiết bị	15 s	Cập nhật trạng thái online trên bảng devices
Khôi phục Wi-Fi	15 s	Tự khởi động lại nếu không kết nối lại được trong thời gian này
Watchdog	60 s	Phát hiện trường hợp tác vụ chính bị treo quá lâu

constrain. Khi thay đổi cảm biến hoặc điều kiện lắp đặt, hai giá trị hiệu chuẩn này cần được đo lại để đảm bảo phần trăm độ ẩm có ý nghĩa thực tế.

Nếu DHT11 đọc lỗi ba lần liên tiếp, firmware ghi cảnh báo lỗi cảm biến lên bảng alertsic. Khi cảm biến hoạt động trở lại, hệ thống gửi thêm một bản ghi phục hồi. Cơ chế này tách lỗi phần cứng khỏi cảnh báo môi trường của cây, giúp ứng dụng có thể hiển thị hai nhóm sự kiện khác nhau.

3.2.4 Thiết kế chế độ điều khiển

Firmware hỗ trợ hai chế độ là manual và auto. Ở chế độ manual, trạng thái quạt và đèn được lấy trực tiếp từ các trường fan_state và light_state trong bảng actuator_states. Ở chế độ auto, firmware tự tính trạng thái dựa trên dữ liệu cảm biến.

Quạt sử dụng hai chốt hysteresis độc lập. Chốt nhiệt độ bật khi nhiệt độ vượt 30°C và chỉ nhả khi nhiệt độ xuống dưới 28°C. Chốt độ ẩm không khí bật khi độ ẩm vượt 80% và chỉ nhả khi độ ẩm xuống dưới 75%. Quạt thực tế bật nếu ít nhất một trong hai chốt đang hoạt động. Cách thiết kế này tránh hiện tượng relay đóng ngắt liên tục khi giá trị dao động quanh một ngưỡng duy nhất. Đèn được bật tự động khi độ ẩm đất nhỏ hơn 60% theo cấu hình hiện tại của firmware.

Mỗi lần trạng thái chấp hành thay đổi ở chế độ auto, firmware ghi một bản ghi vào control_logs. Nếu có thay đổi, trạng thái mới cũng được đồng bộ trở lại actuator_states để ứng dụng nhìn thấy đúng trạng thái thực tế.

3.2.5 Thiết kế hệ thống cảnh báo

Để giảm cảnh báo giả, mã nguồn sử dụng đối tượng trạng thái riêng cho từng loại ngưỡng. Một cảnh báo thường chỉ được kích hoạt khi có hai lần đọc liên tiếp vượt ngưỡng. Sau khi cảnh báo đang hoạt động, hệ thống chỉ xem là đã phục hồi khi giá trị vượt qua một ngưỡng hồi phục riêng, tạo khoảng hysteresis. Nếu trạng thái nguy hiểm kéo dài 15 phút thì mức độ có thể được nâng lên critical; trong khi cảnh báo còn tồn tại, hệ thống nhắc lại sau mỗi 30 phút. Các ngưỡng đang được cài đặt trong firmware được trình bày trong Bảng 3.3.

Bảng 3.2: Ngưỡng cảnh báo môi trường trong firmware

Thông số	Bắt đầu	Phục hồi	Nghiêm trọng ngay	Loại cảnh báo
Độ ẩm đất thấp	$\leq 35\%$	$\geq 40\%$	$\leq 15\%$	Đất quá khô
Độ ẩm đất cao	$\geq 85\%$	$\leq 80\%$	$\geq 95\%$	Đất quá ướt/úng
Nhiệt độ cao	$\geq 35°C$	$\leq 32°C$	$\geq 40°C$	Stress nhiệt
Nhiệt độ thấp	$\leq 10°C$	$\geq 13°C$	$\leq 2°C$	Nhiệt độ thấp/sương giá
Độ ẩm không khí thấp	$\leq 40\%$	$\geq 45\%$	–	Không khí quá khô
Độ ẩm không khí cao	$\geq 85\%$	$\leq 80\%$	–	Nguy cơ nấm bệnh

3.2.6 Khả năng phục hồi và cấu hình thiết bị

Nút reset được thiết kế hai mức. Nếu giữ khoảng 3 giây, thiết bị xóa cấu hình Wi-Fi nhưng giữ mã ghép nối; nếu giữ khoảng 6 giây, firmware xóa cả Wi-Fi và vùng Preferences dùng lưu mã ghép nối. Trong trường hợp mất Wi-Fi, ESP32 nháy LED và thử kết nối lại; nếu quá 15 giây vẫn thất bại thì tự khởi động lại. Ngoài ra watchdog 60 giây được sử dụng để giảm nguy cơ hệ thống treo lâu trong quá trình vận hành.

3.3 Thiết kế cơ sở dữ liệu và backend

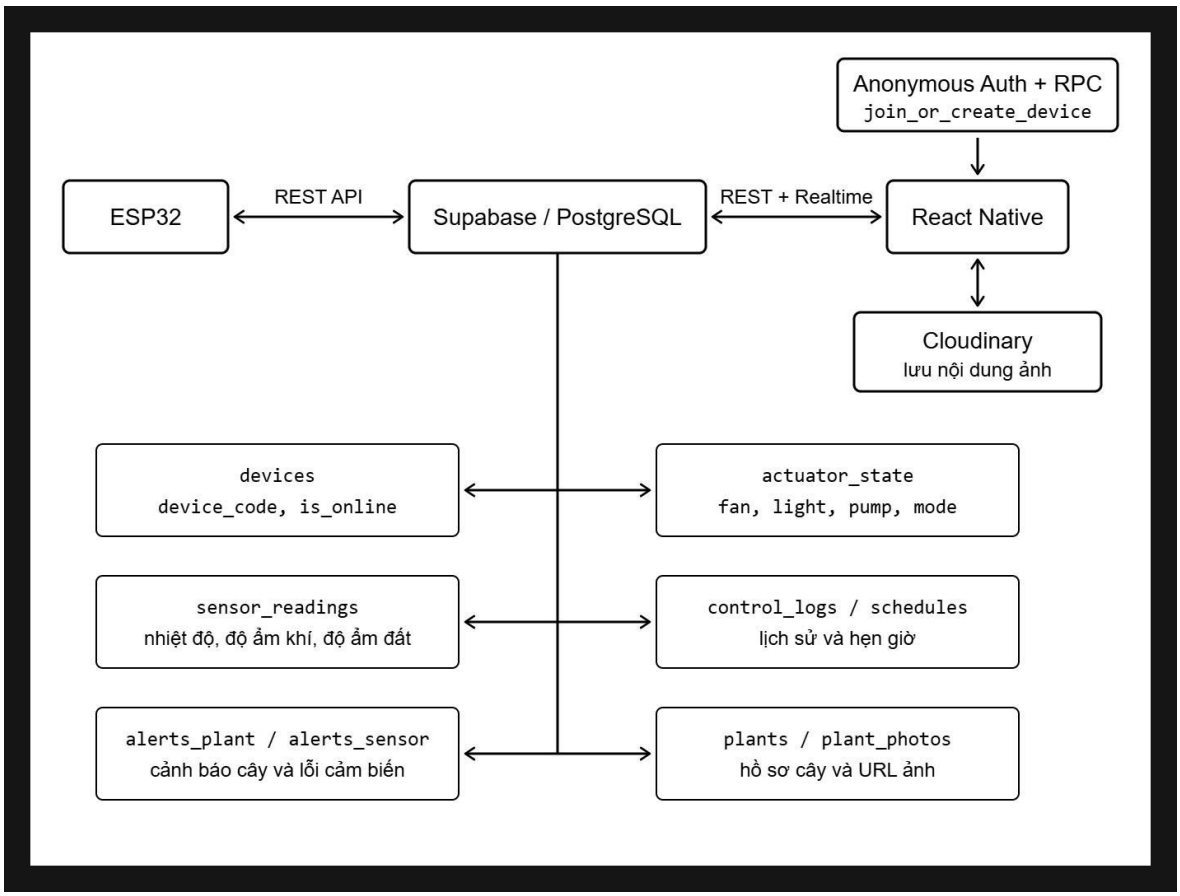
3.3.1 Vai trò của Supabase và Cloudinary

Supabase đóng vai trò backend trung tâm cho dữ liệu có cấu trúc. ESP32 sử dụng REST API để đọc/ghi dữ liệu, còn ứng dụng React Native kết hợp truy vấn REST với Realtime để nhận thay đổi từ PostgreSQL. Trong mã nguồn ứng dụng, phiên người dùng được tạo bằng Supabase Anonymous Auth. Sau đó ứng dụng gọi RPC join_or_create_device với device_code để nhận về device_id. Cơ chế này cho phép tất cả truy vấn ở các màn hình được lọc theo thiết bị đang quản lý.

Ảnh cây không được lưu trực tiếp vào PostgreSQL. PlantsScreen chụp ảnh bằng expo-image-picker, tải file lên Cloudinary và nhận về URL HTTPS. URL này được ghi vào trường storage_path của bảng plant_photos. Cách tách ảnh khỏi cơ sở dữ liệu quan hệ giúp bảng dữ liệu chỉ lưu metadata và liên kết đến file ảnh.

3.3.2 Các bảng dữ liệu được sử dụng

Từ các truy vấn trong firmware và ứng dụng, hệ thống hiện sử dụng các bảng và trường chính như Bảng 3.4. Bảng này mô tả đúng các trường được mã nguồn tham chiếu; những cột khác có thể tồn tại trong cơ sở dữ liệu nhưng không được suy đoán khi không xuất hiện trong source.



Hình 3.3: Các bảng dữ liệu được sử dụng

3.3.3 Trao đổi dữ liệu Realtime

Ở màn hình Trang chủ, ứng dụng đăng ký kênh Realtime đối với sự kiện INSERT trên bảng sensor_readings, có điều kiện lọc device_id. Khi một bản ghi mới xuất hiện, callback cập nhật latest và nối dữ liệu vào lịch sử 24 giờ. Ở màn hình Điều khiển, một kênh khác lắng nghe mọi thay đổi trên actuator_states và các bản ghi mới trên control_logs. Nhờ vậy trạng thái hiển thị trên điện thoại có thể thay đổi theo dữ liệu thực tế mà không cần gửi truy vấn liên tục.

Phần cảnh báo được đọc từ hai bảng alertsic và alertsplant. Ứng dụng gộp hai nguồn, sắp xếp theo thời gian giảm dần và hiển thị tối đa 20 cảnh báo mới nhất. Cách tách hai bảng giúp phân biệt cảnh báo lỗi thiết bị với cảnh báo điều kiện môi trường của cây.

3.3.4 Ràng buộc giữa thiết bị và ứng dụng

Firmware lấy device_code từ cổng cấu hình Wi-Fi, trong khi ứng dụng hiện đặt một hằng DEVICE_CODE trong session.ts. Hai giá trị phải trùng nhau thì ESP32 và ứng dụng mới truy cập cùng một device_id. Do đó ở phiên bản triển khai thực tế, mã ghép nối nên được người dùng nhập hoặc quét trong ứng dụng thay vì cố định trong source.

Ứng dụng có giao diện tạo, bật/tắt và xóa các bản ghi schedules. Tuy nhiên ba file mã nguồn được cung cấp không chứa một tiến trình backend hoặc đoạn firmware đọc bảng lịch

và thực thi theo thời gian. Vì vậy chương này chỉ xác nhận chức năng lưu và quản lý lịch ở phía ứng dụng; cơ chế thực thi lịch cần được bổ sung bằng Edge Function, cron job, trigger phù hợp hoặc xử lý trong firmware nếu muốn lịch hoạt động độc lập.

Về bảo mật, mã nguồn có chứa các khóa truy cập dịch vụ. Báo cáo không trình bày giá trị khóa. Khi triển khai thực tế, khóa quyền cao không nên được nhúng trực tiếp trong firmware hoặc ứng dụng; nên sử dụng quyền tối thiểu, RLS và/hoặc một lớp API trung gian để giới hạn phạm vi truy cập.

3.4 Thiết kế mô hình học sâu

3.4.1 Tổ chức dữ liệu và các lớp phân loại

Notebook huấn luyện tổ chức bài toán thành tám lớp theo thứ tự cố định. Thứ tự này đồng thời được lưu cùng model và phải được giữ nguyên khi tích hợp ứng dụng:

1) angular_leafspot – đốm lá góc cạnh;
2) anthracnose_fruit_rot – thán thư trên quả;
3) blossom_blight – cháy hoa;
4) gray_mold – mốc xám;
5) healthy – khỏe mạnh;
6) leaf_spot – đốm lá;
7) powdery_mildew_fruit – phấn trắng trên quả;
8) powdery_mildew_leaf – phấn trắng trên lá.

Trước khi chia tập, notebook kiểm tra ảnh, sửa hướng EXIF, chuyển về RGB và phát hiện ảnh trùng chính xác bằng hash pixel. Tổng cộng 41 ảnh trùng chính xác được loại và không có ảnh lỗi, để lại 2915 ảnh hợp lệ. Tập dữ liệu được chia theo tỷ lệ 70% train, 15% validation và 15% test, với seed 42. Phân bố thực tế được trình bày trong Bảng 3.5.

Notebook hiện chưa sử dụng group_id, do đó phép chia chưa đảm bảo các ảnh gần trùng hoặc các ảnh của cùng một cây luôn nằm trong cùng một tập. Đây là giới hạn cần lưu ý khi diễn giải kết quả kiểm thử; nếu có metadata theo cây hoặc theo phiên chụp, có thể bổ sung groups.csv để thực hiện group-aware split.

3.4.2 Tiền xử lý và tăng cường dữ liệu

Ảnh được giải mã RGB và resize bằng nội suy bilinear về $224 \times 224 \times 3$. Backbone được khởi tạo với include_preprocessing=True, vì vậy dữ liệu đầu vào được giữ ở kiểu float32 trong miền 0–255 và không chia thêm cho 255 ở bên ngoài model. Đây là điều kiện quan

Bảng 3.3: Phân bố dữ liệu sau khi kiểm tra và loại ảnh trùng

Lớp	**Tổng**	**Train**	**Val.**	**Test**
angular_leafspot	435	305	65	65
anthracnose_fruit_rot	97	68	15	14
blossom_blight	208	146	31	31
gray_mold	477	334	72	71
healthy	456	319	69	68
leaf_spot	574	402	86	86
powdery_mildew_fruit	135	95	20	20
powdery_mildew_leaf	533	373	80	80
Tổng	2915	2042	438	435

trọng khi triển khai TFLite: ứng dụng phải tạo tensor đầu vào tương thích với đúng miền giá trị này.

Tăng cường dữ liệu chỉ áp dụng trên tập train, gồm lật ngang/dọc, xoay ngẫu nhiên mức 0,10, zoom 0,10, thay đổi tương phản 0,10 và độ sáng 0,10. Giá trị sau augmentation được giới hạn lại trong miền 0–255. Validation và test không được augmentation để giữ vai trò đánh giá độc lập.

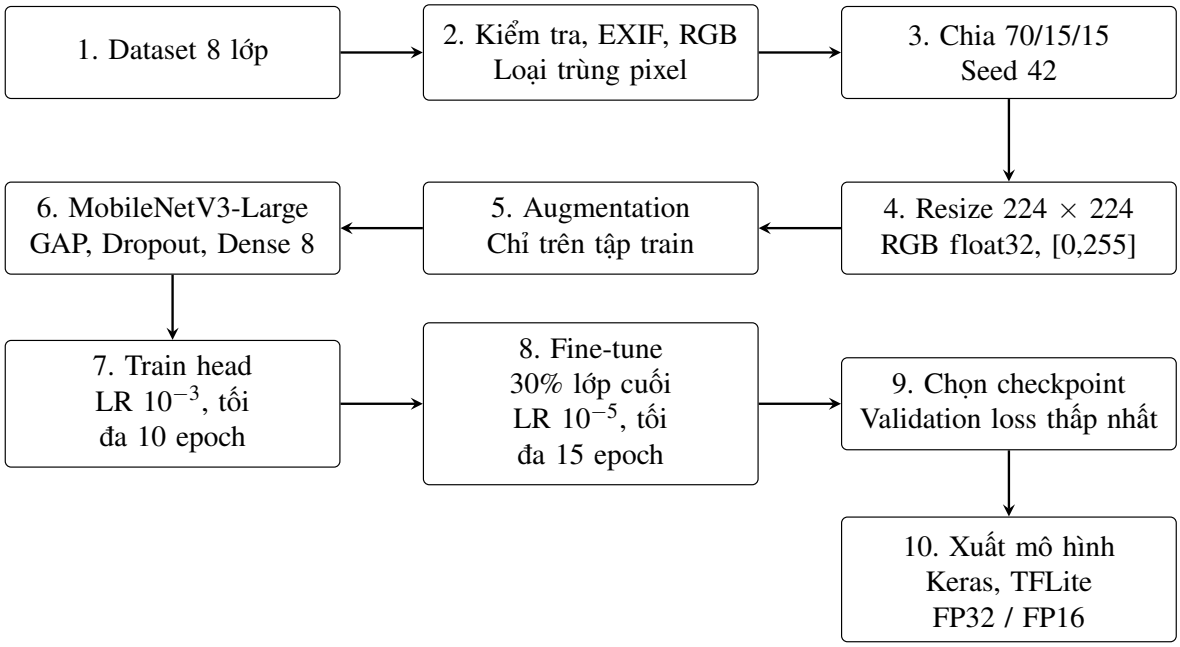
Do số lượng ảnh giữa các lớp không cân bằng, mã huấn luyện tính class_weight theo tần suất lớp trên tập train. Trọng số này được truyền vào model.fit, giúp các lớp ít mẫu đóng góp nhiều hơn vào hàm mất mát.

3.4.3 Kiến trúc và chiến lược huấn luyện

Backbone sử dụng MobileNetV3Large với $\alpha = 1.0$, cấu hình non-minimalistic, trọng số ImageNet, include_top=False và Global Average Pooling. Đầu phân loại của đề tài gồm Dropout 0,30 và lớp Dense tám nút với Softmax. Hàm mất mát là SparseCategoricalCrossentropy và bộ tối ưu Adam.

Quá trình huấn luyện được chia thành hai giai đoạn. Giai đoạn đầu đóng băng toàn bộ backbone và huấn luyện đầu phân loại với learning rate 10^{-3} trong tối đa 10 epoch. Giai đoạn thứ hai mở 30% số lớp cuối của backbone nhưng tiếp tục giữ các lớp BatchNormalization ở trạng thái frozen; learning rate giảm xuống 10^{-5} và huấn luyện tối đa 15 epoch. Ở cả hai giai đoạn, checkpoint được theo dõi bằng validation loss. Mã nguồn sử dụng EarlyStopping với patience bằng 5, ReduceLROnPlateau với hệ số 0,5 và ModelCheckpoint để lưu trọng số tốt nhất.

Model cuối cùng được chọn theo validation loss nhỏ nhất giữa checkpoint của giai đoạn huấn luyện head và checkpoint fine-tuning. Tập test chỉ được dùng sau khi chọn xong model. Cách tách này giúp hạn chế việc vô tình tối ưu siêu tham số trực tiếp trên tập test.



Hình 3.4: Quy trình chuẩn bị dữ liệu, huấn luyện và xuất mô hình

3.4.4 Định dạng triển khai và giao diện dữ liệu cho ứng dụng

Sau huấn luyện, mã nguồn lưu model Keras và có tùy chọn xuất cả TFLite FP32 và TFLite FP16. Cả hai bản TFLite được thiết kế với đầu vào float32, ảnh RGB 224 × 224 và miền pixel 0–255. Đầu ra là vector tám xác suất theo đúng thứ tự lớp đã nêu ở trên.

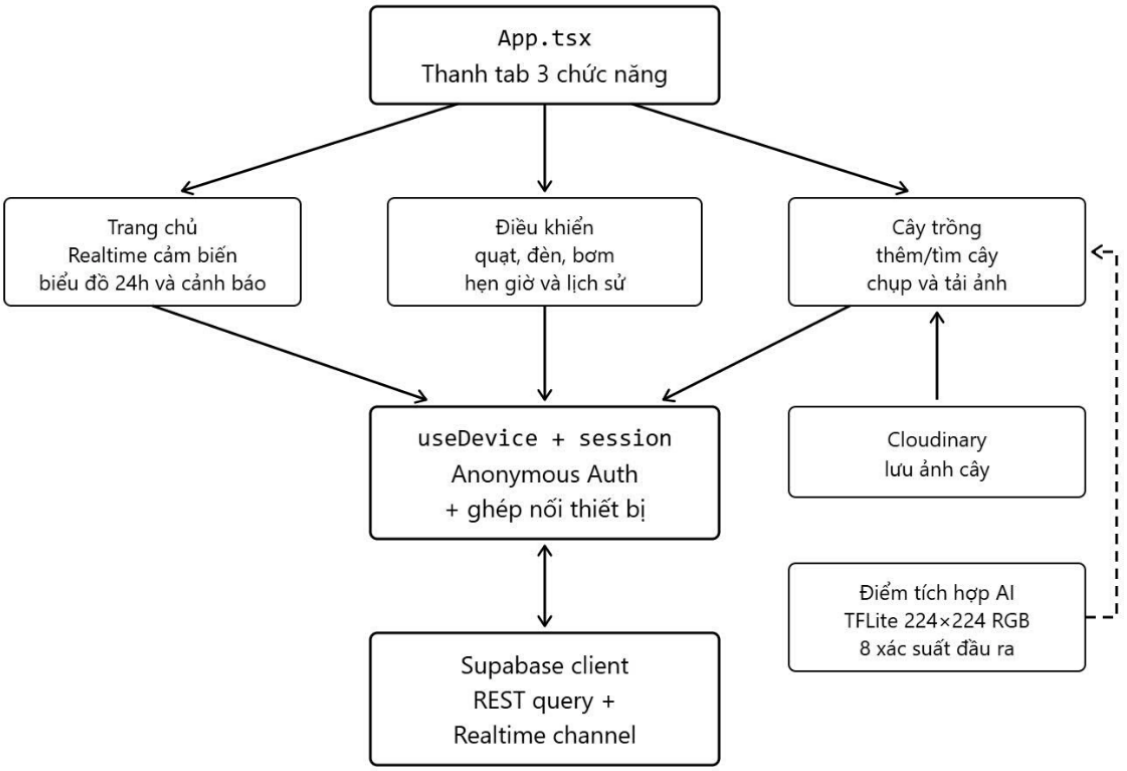
Trong ứng dụng React Native, mô-đun suy luận thực hiện tuần tự: lấy ảnh camer- a/thư viện, sửa hướng ảnh nếu cần, resize về 224 × 224, chuyển sang RGB float32 trong miền 0–255, chạy TFLite, lấy chỉ số xác suất lớn nhất và ánh xạ chỉ số đó về đúng nhãn. Không được thay đổi thứ tự nhãn và không được chia ảnh cho 255 nếu file TFLite vẫn giữ preprocessing như model đã xuất.

Trong notebook, file TFLite đã được xuất nhưng chưa được đánh giá lại trên toàn bộ tập test bằng runtime TFLite. Vì vậy việc so sánh đầu ra Keras và TFLite cần được thực hiện trước khi xem hai định dạng là hoàn toàn tương đương trong ứng dụng.

3.5 Thiết kế giao diện ứng dụng (UI/UX)

3.5.1 Cấu trúc điều hướng

Ứng dụng được xây dựng bằng React Native theo cấu trúc ba tab. App.tsx quản lý trạng thái activeTab và chuyển đổi giữa ba màn hình: Trang chủ, Điều khiển và Cây trồng. Thanh tab được xây dựng trực tiếp bằng các component React Native và có hiệu ứng animation khi người dùng chuyển tab.



Hình 3.5: Cấu trúc chức năng của ứng dụng React Native

Hook useDevice được dùng chung cho các màn hình. Hook gọi ensureDeviceAccess, tạo phiên Anonymous Auth nếu chưa có, sau đó gọi RPC để lấy device_id. Nhờ vậy mỗi màn hình chỉ cần nhận deviceId rồi thực hiện các truy vấn được lọc theo thiết bị.

3.5.2 Màn hình Trang chủ

Màn hình HomeScreen có hai nhóm chức năng: giám sát thời gian gần thực và xem thống kê lịch sử. Khi mở màn hình, ứng dụng truy vấn các bản ghi sensor_readings trong 24 giờ gần nhất và hiển thị giá trị mới nhất của nhiệt độ, độ ẩm không khí và độ ẩm đất. Dữ liệu được lấy mẫu đều để vẽ biểu đồ bằng react-native-chart-kit, tránh số điểm quá dày trên màn hình nhỏ.

Ngoài biểu đồ 24 giờ, người dùng có thể chọn khoảng 7 ngày, 30 ngày hoặc 3 tháng. Ứng dụng truy vấn dữ liệu tương ứng và tạo ba biểu đồ riêng cho nhiệt độ, độ ẩm không khí và độ ẩm đất. Màn hình đồng thời đọc cảnh báo từ alertsic và alertsplant, gộp hai nguồn theo thời gian và hiển thị lịch sử cảnh báo mới nhất.

Kênh Realtime sensor-readings-<deviceId> lắng nghe sự kiện INSERT. Khi ESP32 gửi một bản đo mới, giao diện cập nhật latest và lịch sử 24 giờ mà không cần người dùng tải lại trang.

3.5.3 Màn hình Điều khiển

ControlScreen hiển thị ba công tắc cho quạt, đèn và máy bơm. Khi người dùng thay đổi công tắc, ứng dụng upsert bảng actuator_states, đặt mode='manual' và ghi một bản ghi vào control_logs. Màn hình cũng đăng ký Realtime cho actuator_states và control_logs để đồng bộ trạng thái và lịch sử điều khiển.

Phần hẹn giờ cho phép chọn thiết bị, hành động bật/tắt, thời gian định dạng HH:MM và các ngày trong tuần. Bản ghi được lưu vào bảng schedules, đồng thời người dùng có thể bật/tắt hoặc xóa lịch. Như đã nêu ở phần backend, source được cung cấp chưa có tiến trình thực thi lịch, nên giao diện này hiện chủ yếu chịu trách nhiệm tạo và quản lý cấu hình lịch.

Mặc dù UI có máy bơm, firmware hiện không có RELAY_PUMP. Vì vậy trong phiên bản phần cứng hiện tại, chỉ hai công tắc quạt và đèn có đường điều khiển thực tế đến ESP32; công tắc máy bơm cần được bổ sung chân relay và logic firmware nếu muốn sử dụng.

3.5.4 Màn hình Cây trồng

PlantsScreen cho phép thêm cây với các thông tin: tên cây, mã cây sinh tự động, ngày trồng, thuốc trừ sâu và loại phân bón. Ảnh cây được chụp bằng camera thông qua expo-image-picker; trước khi mở camera, ứng dụng yêu cầu quyền truy cập camera. Ảnh được cắt theo tỷ lệ 1:1 và nén với chất lượng 0,7 trước khi upload. Sau khi tạo bản ghi cây trên Supabase, nếu có ảnh, ứng dụng tải ảnh lên Cloudinary và nhận secure_url. URL này được lưu trong plant_photos. Khi tải danh sách cây, ứng dụng lấy ảnh mới nhất của từng cây để hiển thị thumbnail. Người dùng có thể tìm cây theo mã cây và lọc theo khoảng ngày trồng.

3.5.5 Thiết kế màn hình nhận diện bệnh

Ứng dụng đã tích hợp mô-đun TFLite để hỗ trợ nhận diện bệnh từ ảnh cây trồng. Luồng xử lý của chức năng nhận diện tuân theo giao diện dữ liệu của mô hình như sau:

1) Người dùng chọn ảnh từ camera hoặc thư viện.

2) Ảnh được chuẩn hóa hướng và resize về 224×224 RGB.

3) Tensor được giữ ở kiểu float32, miền 0–255.

4) TFLite trả về vector xác suất tám lớp.

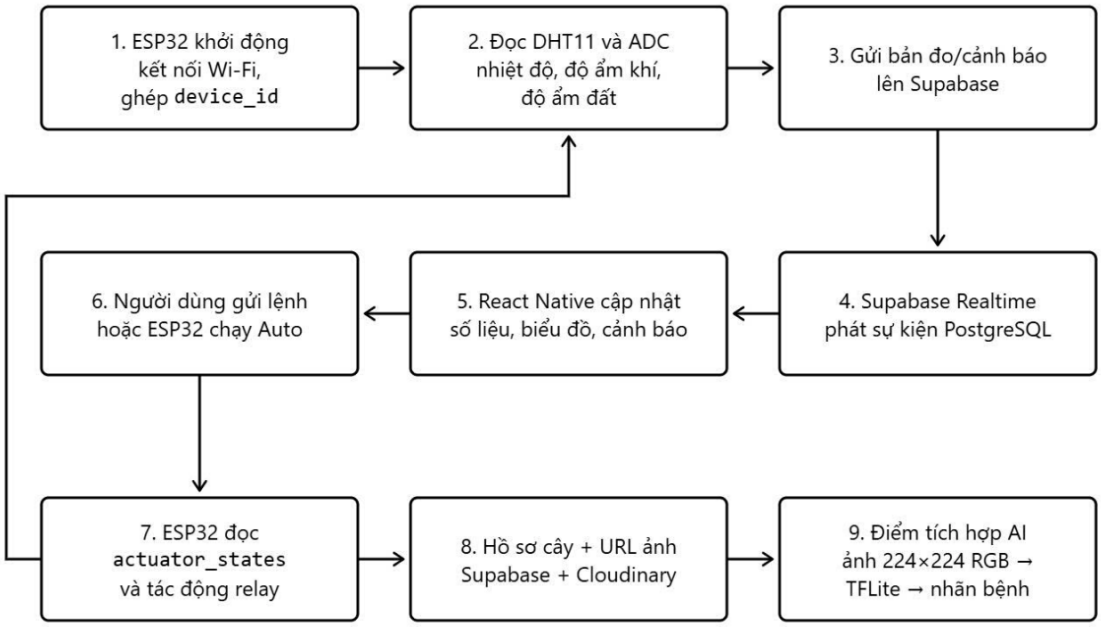
5) Ứng dụng chọn lớp có xác suất lớn nhất, ánh xạ sang tên tiếng Việt và hiển thị cùng điểm Softmax của lớp dự đoán.

6) Điểm Softmax không phải độ chắc chắn đã được hiệu chuẩn. Mô hình chưa có lớp unknown và không bảo đảm phát hiện ảnh ngoài phạm vi dâu tây; người dùng cần kiểm tra chất lượng, nội dung ảnh và có thể chụp lại khi kết quả chưa phù hợp.

Mô-đun nhận diện hoạt động trong ứng dụng di động và sử dụng ảnh do người dùng cung cấp. Kết quả phân loại hỗ trợ theo dõi tình trạng cây trồng, không tự động thay thế việc kiểm tra thực tế hoặc quyết định điều khiển thiết bị.

3.6 Thiết kế luồng hoạt động

Luồng hoạt động tổng thể được xây dựng theo hướng dữ liệu cảm biến đi từ ESP32 lên backend, còn lệnh điều khiển đi theo chiều ngược lại. Ứng dụng không kết nối trực tiếp đến ESP32 mà giao tiếp qua Supabase. Cách này giúp điện thoại và thiết bị phần cứng không cần ở cùng mạng nội bộ, đồng thời cho phép lưu lịch sử và đồng bộ trạng thái qua cơ sở dữ liệu.



Hình 3.6: Luồng vận hành tổng thể của hệ thống

3.6.1 Luồng giám sát môi trường

ESP32 đọc cảm biến mỗi 10 giây. Nếu DHT11 hợp lệ, firmware gửi một bản ghi mới lên sensor_readings. Supabase phát sự kiện thay đổi PostgreSQL tới kênh Realtime mà ứng dụng đã đăng ký. HomeScreen nhận payload, cập nhật số liệu mới nhất và nối điểm dữ liệu vào biểu đồ 24 giờ. Song song, ESP32 đánh giá các ngưỡng cảnh báo và ghi sự kiện vào bảng tương ứng nếu phát hiện trạng thái nguy hiểm hoặc trạng thái đã phục hồi.

3.6.2 Luồng điều khiển thủ công

Khi người dùng bật/tắt quạt hoặc đèn, ứng dụng cập nhật actuator_states với mode='manual' và ghi control_logs. ESP32 kiểm tra bảng trạng thái mỗi 3 giây. Nếu mode là manual, firmware đặt mức GPIO tương ứng và cập nhật biến trạng thái nội bộ. Realtime giúp giao

diện nhận lại thay đổi từ cơ sở dữ liệu, nhờ đó trạng thái hiển thị thống nhất giữa các phiên ứng dụng.

3.6.3 Luồng điều khiển tự động

Khi mode là auto, ESP32 không dùng trực tiếp giá trị công tắc từ ứng dụng để quyết định relay mà tính trạng thái mới từ nhiệt độ, độ ẩm không khí và độ ẩm đất. Hysteresis riêng cho nhiệt độ và độ ẩm không khí giúp quạt ổn định hơn. Sau khi relay thay đổi, firmware ghi log và cập nhật trạng thái về Supabase để giao diện hiển thị đúng kết quả điều khiển tự động.

3.6.4 Luồng quản lý cây và ảnh

Người dùng tạo hồ sơ cây trong PlantsScreen. Bản ghi metadata được tạo trước trên Supabase. Nếu người dùng có chụp ảnh, ảnh được upload lên Cloudinary; sau khi nhận URL, ứng dụng tạo bản ghi plant_photos trỏ đến cây vừa tạo. Danh sách cây sau đó ghép metadata từ Supabase với URL ảnh mới nhất để hiển thị.

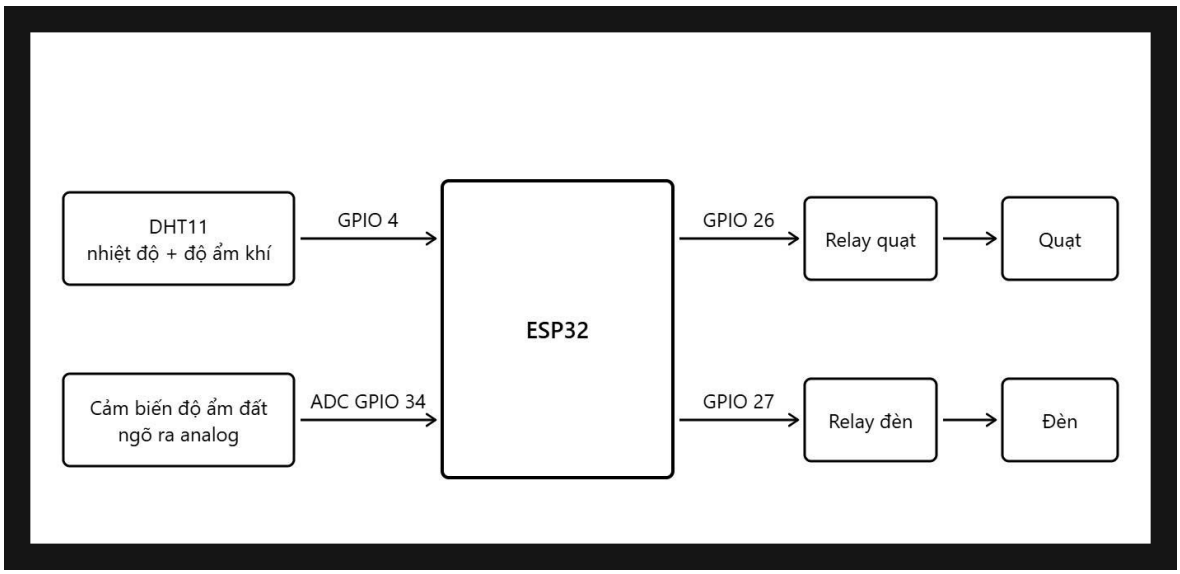
3.6.5 Luồng suy luận AI

Model được huấn luyện độc lập trên môi trường TensorFlow/Keras và xuất thành TFLite. Mô-đun suy luận đã được tích hợp vào ứng dụng để xử lý ảnh trực tiếp trên điện thoại, không cần gửi ảnh đến máy chủ AI. Ứng dụng tiếp nhận ảnh, sửa hướng nếu cần, resize về 224×224, chuyển thành tensor RGB float32 trong miền 0–255 và chạy mô hình. Vector đầu ra được ánh xạ về đúng thứ tự tám lớp để hiển thị nhãn dự đoán cùng điểm Softmax. Việc tích hợp mô-đun không thay thế yêu cầu đánh giá độ chính xác, thời gian suy luận và mức sử dụng tài nguyên trên thiết bị di động thực tế.

3.7 Thiết kế phần cứng

3.7.1 Cấu trúc phần cứng

Khối phần cứng được thiết kế xoay quanh ESP32. DHT11 cung cấp dữ liệu nhiệt độ và độ ẩm không khí qua GPIO 4. Cảm biến độ ẩm đất đưa tín hiệu analog vào GPIO 34. Hai ngõ ra GPIO 26 và GPIO 27 điều khiển relay quạt và relay đèn. GPIO 0 được dùng làm nút reset, còn GPIO 2 điều khiển LED trạng thái.



Hình 3.7: Sơ đồ khối kết nối phần cứng theo firmware

Firmware đặt RELAY_LOW_TRIGGER=false, nghĩa là cấu hình hiện tại giả định relay hoạt động theo mức logic cao. Khi sử dụng module relay khác có mức kích thấp, cần thay đổi tham số này hoặc đảo logic điều khiển để tránh trạng thái bật/tắt ngược với mong muốn.

3.7.2 Nguyên tắc lắp đặt và vận hành

Cảm biến độ ẩm đất cần được đặt ở vị trí đại diện cho vùng rễ và hiệu chuẩn lại nếu thay đầu dò. DHT11 nên được bố trí tránh nước tưới trực tiếp và tránh đặt sát nguồn nhiệt để giảm sai lệch. Relay phải được chọn theo điện áp và dòng tải của quạt/đèn; phần công suất cần tách an toàn khỏi phần logic của ESP32. Nguồn cấp cho ESP32 và module relay cần ổn định để tránh reset khi tải chuyển trạng thái.

Nút reset hai mức và LED trạng thái hỗ trợ vận hành tại hiện trường mà không cần kết nối cáp lập trình. Khi đổi mạng Wi-Fi, người dùng có thể xóa riêng cấu hình mạng mà vẫn giữ mã ghép nối; khi cần đưa thiết bị về trạng thái ban đầu, thao tác reset toàn bộ sẽ xóa cả cấu hình Wi-Fi và mã thiết bị.

3.7.3 Phạm vi phần cứng hiện tại và khả năng mở rộng

Theo firmware cung cấp, phần cứng hiện thực gồm quạt và đèn. Ứng dụng đã chuẩn bị trường dữ liệu và UI cho máy bơm, nhưng không có chân GPIO, relay hoặc nhánh điều khiển máy bơm trong firmware. Để bổ sung máy bơm cần khai báo một chân relay mới, mở rộng hàm cập nhật trạng thái, xử lý manual/auto và đồng bộ log tương tự quạt/đèn.

Ngoài ra, cấu trúc backend và ứng dụng cho phép mở rộng nhiều thiết bị thông qua device_id. Khi triển khai nhiều node ESP32, mỗi node cần một mã ghép nối riêng và các truy vấn của ứng dụng phải tiếp tục lọc theo thiết bị đang được chọn. Đây là cơ sở để hệ

thống có thể phát triển từ mô hình thử nghiệm một khu vực trồng sang nhiều khu vực hoặc nhiều luống dâu trong tương lai.

CHƯƠNG 4

KẾT QUẢ

Chương này trình bày các kết quả đạt được sau quá trình xây dựng mô hình thử nghiệm quản lý và giám sát nông trại dâu. Nội dung được tổng hợp từ chương trình nhúng trên ESP32, ứng dụng di động React Native, cấu trúc dữ liệu Supabase và mô hình MobileNetV3-Large đã được huấn luyện và xuất sang TensorFlow Lite. Kết quả được xem xét theo bốn nhóm chính gồm mô hình học sâu, giao diện ứng dụng di động, phần cứng IoT và khả năng tích hợp giữa các thành phần.

Trong phiên bản mã nguồn mới nhất, mô-đun nhận diện bệnh bằng TensorFlow Lite đã được tích hợp trực tiếp vào ứng dụng. Vì vậy, ngoài các chức năng giám sát và điều khiển môi trường, ứng dụng còn có khả năng nhận ảnh từ camera hoặc thư viện và thực hiện suy luận cục bộ trên điện thoại.

4.1 Kết quả huấn luyện mô hình học sâu

4.1.1 Kết quả chuẩn bị dữ liệu

Bài toán nhận diện bệnh được tổ chức thành tám lớp gồm bảy nhóm bệnh và một lớp cây khỏe mạnh. Thứ tự lớp được giữ cố định từ quá trình huấn luyện đến khi triển khai trên ứng dụng nhằm bảo đảm chỉ số đầu ra của mô hình được ánh xạ đúng sang tên bệnh. Các lớp được sử dụng gồm:

- `angular_leafspot`: đốm lá góc cạnh;
- `anthracnose_fruit_rot`: thán thư trên quả;
- `blossom_blight`: cháy hoa;
- `gray_mold`: mốc xám;
- `healthy`: khỏe mạnh;
- `leaf_spot`: đốm lá;
- `powdery_mildew_fruit`: phấn trắng trên quả;
- `powdery_mildew_leaf`: phấn trắng trên lá.

Trước khi chia tập dữ liệu, notebook thực hiện kiểm tra ảnh, sửa hướng ảnh theo thông tin EXIF, chuyển ảnh về RGB và phát hiện ảnh trùng hoàn toàn bằng giá trị hash của pixel. Tổng cộng 41 ảnh trùng được loại bỏ và không phát hiện ảnh lỗi, còn lại 2915 ảnh hợp lệ. Dữ liệu sau đó được chia theo tỷ lệ 70% cho tập train, 15% cho tập validation và 15% cho tập test với seed bằng 42. Phân bố cụ thể được trình bày trong Bảng 4.1.

Bảng 4.1: Phân bố dữ liệu sử dụng cho quá trình huấn luyện và đánh giá

Lớp	**Tổng**	**Train**	**Validation**	**Test**
angular_leafspot	435	305	65	65
anthracnose_fruit_rot	97	68	15	14
blossom_blight	208	146	31	31
gray_mold	477	334	72	71
healthy	456	319	69	68
leaf_spot	574	402	86	86
powdery_mildew_fruit	135	95	20	20
powdery_mildew_leaf	533	373	80	80
Tổng	**2915**	**2042**	**438**	**435**

Có thể thấy số lượng ảnh giữa các lớp chưa cân bằng. Lớp `anthracnose_fruit_rot` có 97 ảnh, trong khi lớp `leaf_spot` có 574 ảnh. Để giảm ảnh hưởng của sự chênh lệch này, quá trình huấn luyện tính `class_weight` từ tần suất của từng lớp trong tập train và truyền các trọng số này vào hàm `model.fit`.

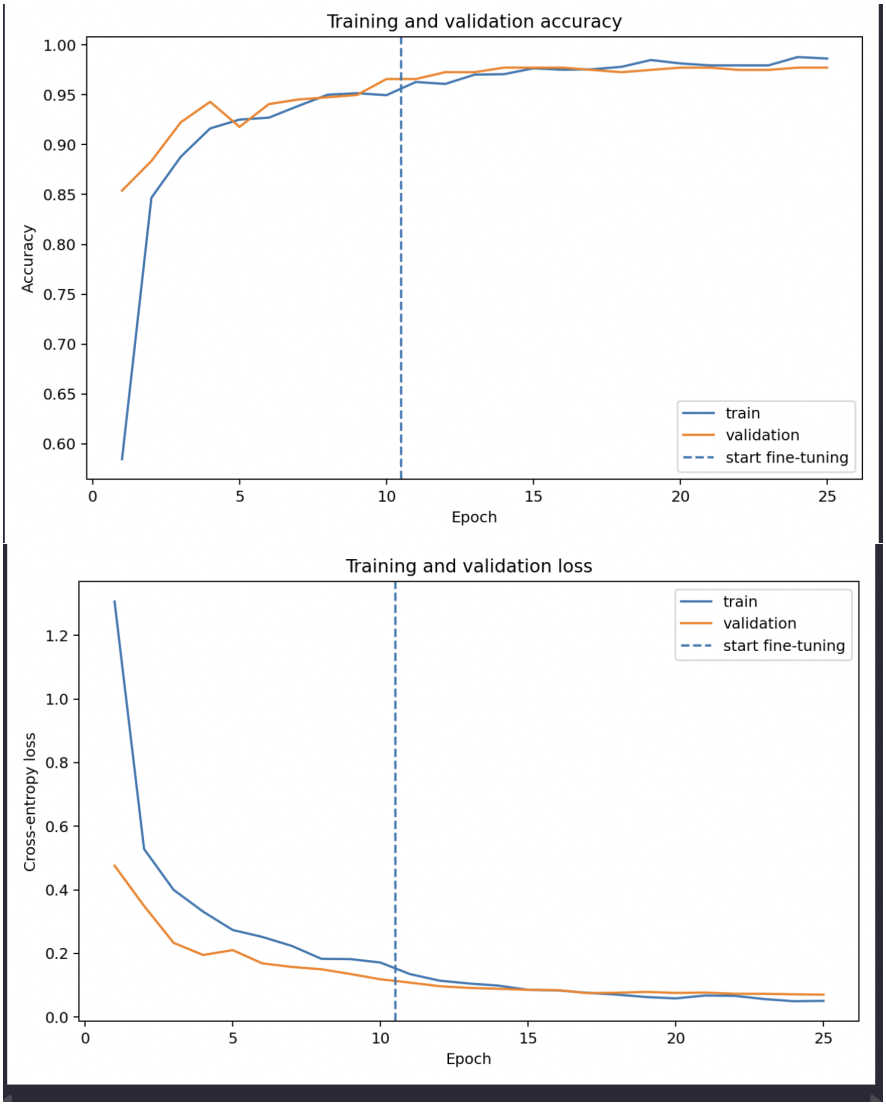
Ảnh đầu vào được resize bằng nội suy bilinear về kích thước $224 \times 224 \times 3$. Backbone MobileNetV3-Large được khởi tạo với preprocessing bên trong mô hình, do đó tensor đầu vào giữ ở kiểu `float32` và miền giá trị pixel từ 0 đến 255, không chia thêm cho 255 ở bên ngoài. Tăng cường dữ liệu chỉ được áp dụng cho tập train, gồm lật ảnh, xoay ngẫu nhiên, zoom, thay đổi độ tương phản và độ sáng. Tập validation và test không sử dụng augmentation để giữ vai trò đánh giá độc lập.

4.1.2 Kết quả xây dựng và huấn luyện mô hình

Mô hình sử dụng MobileNetV3-Large với trọng số khởi tạo từ ImageNet, bỏ phần phân loại mặc định của kiến trúc và sử dụng Global Average Pooling. Phần phân loại của đề tài gồm lớp Dropout với tỷ lệ 0,30 và lớp Dense tám nút với hàm Softmax. Hàm mất mát được sử dụng là Sparse Categorical Crossentropy và bộ tối ưu là Adam.

Quá trình huấn luyện được chia thành hai giai đoạn. Trong giai đoạn đầu, toàn bộ backbone được đóng băng và chỉ phần đầu phân loại được huấn luyện với learning rate 10^{-3} trong tối đa 10 epoch. Trong giai đoạn fine-tuning, 30% số lớp cuối của backbone được mở để tiếp tục huấn luyện, các lớp Batch Normalization vẫn được giữ ở trạng thái đóng băng và learning rate giảm xuống 10^{-5}. Giai đoạn này được huấn luyện tối đa 15 epoch.

Để hạn chế overfitting và lựa chọn mô hình tốt hơn, notebook sử dụng Early Stopping với patience bằng 5, ReduceLROnPlateau với hệ số 0,5 và Model Checkpoint theo dõi validation loss. Mô hình cuối cùng được chọn từ checkpoint có validation loss nhỏ nhất giữa hai giai đoạn. Tập test chỉ được sử dụng sau khi quá trình lựa chọn mô hình đã hoàn tất.



Hình 4.1: Biểu đồ quá trình huấn luyện và fine-tuning mô hình

Kết quả huấn luyện xác nhận mô hình có thể được lưu ở định dạng Keras và chuyển đổi sang TensorFlow Lite. Source ứng dụng hiện chứa cả phiên bản TFLite FP32 và FP16. Kích thước tệp FP32 xấp xỉ 11,37 MiB, trong khi bản FP16 xấp xỉ 5,73 MiB. Việc có cả hai phiên bản tạo điều kiện so sánh giữa kích thước mô hình và hiệu năng suy luận trên thiết bị di động.

4.1.3 Đánh giá mô hình trên tập kiểm thử

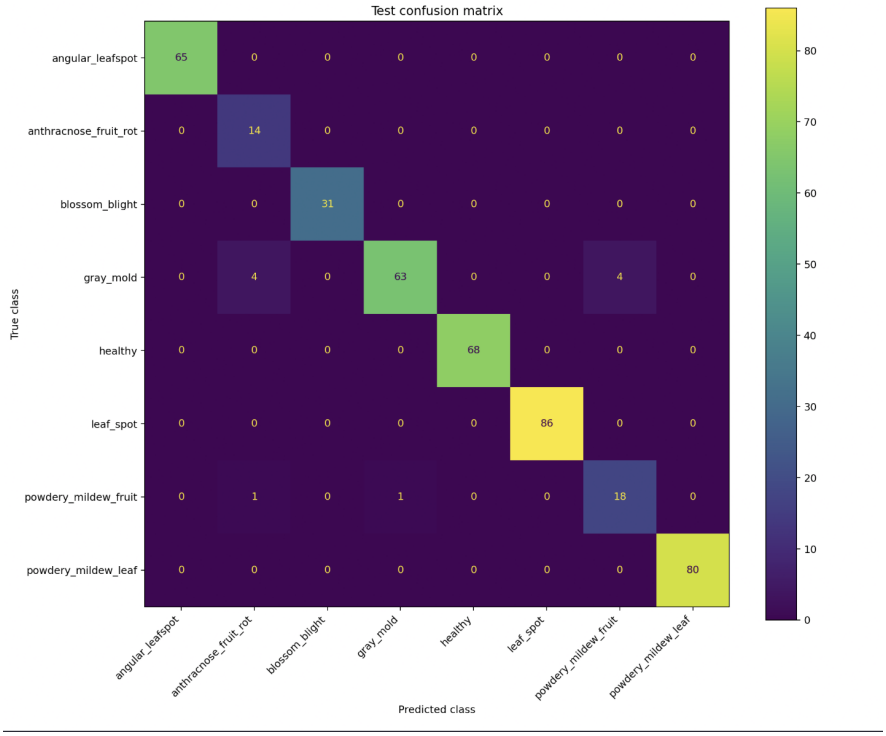
Tập test gồm 435 ảnh và được giữ riêng cho bước đánh giá sau cùng. Các chỉ số phù hợp cho bài toán bao gồm test loss, accuracy, precision, recall và F1-score. Ngoài chỉ số tổng thể, confusion matrix và classification report cần được sử dụng để xem xét khả năng nhận diện ở từng lớp, đặc biệt đối với các lớp có số lượng ảnh thấp.

```
}
                        precision    recall  f1-score   support

     angular_leafspot     1.0000    1.0000    1.0000        65
anthracnose_fruit_rot     0.7368    1.0000    0.8485        14
       blossom_blight     1.0000    1.0000    1.0000        31
            gray_mold     0.9844    0.8873    0.9333        71
              healthy     1.0000    1.0000    1.0000        68
            leaf_spot     1.0000    1.0000    1.0000        86
   powdery_mildew_fruit     0.8182    0.9000    0.8571        20
    powdery_mildew_leaf     1.0000    1.0000    1.0000        80

             accuracy                         0.9770       435
```

Hình 4.2: Kết quả đánh giá mô hình MobileNetV3-Large trên tập test



Hình 4.3: Ma trận nhầm lẫn của mô hình trên tập kiểm thử

Khi phân tích kết quả, cần lưu ý rằng notebook hiện chưa sử dụng `group_id` để chia dữ liệu theo cây hoặc theo phiên chụp. Do đó, mặc dù các ảnh trùng hoàn toàn đã được loại bỏ trước khi chia tập, vẫn có khả năng tồn tại các ảnh gần giống nhau của cùng một cây ở các tập khác nhau. Đây là giới hạn cần được xem xét khi diễn giải độ chính xác trên tập test.

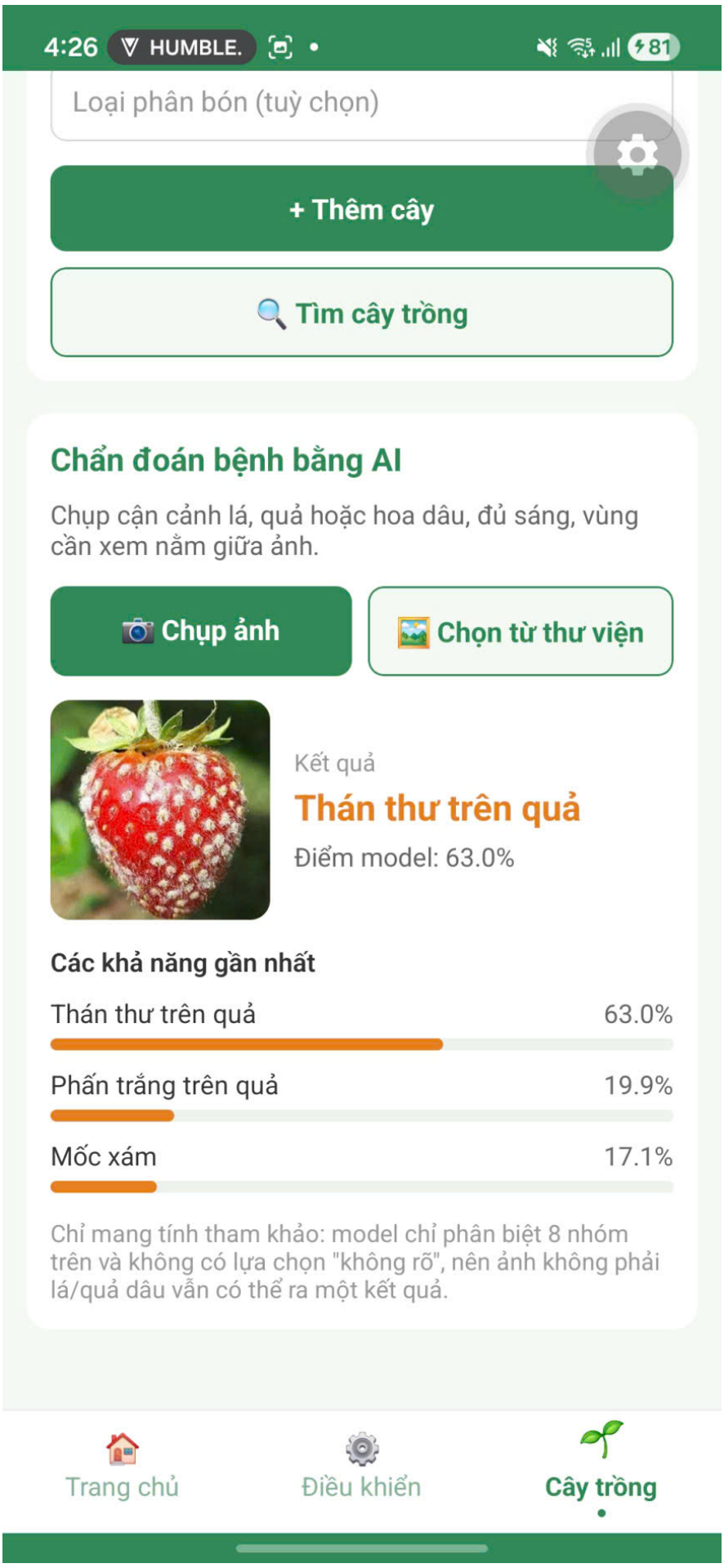
4.1.4 Kết quả tích hợp mô hình TensorFlow Lite

Phiên bản ứng dụng hiện tại đã tích hợp mô-đun nhận diện bệnh trong component `DiseaseScanner`. Ứng dụng sử dụng thư viện `react-native-fast-tflite` và tải mô hình FP32 từ thư mục `assets/model`. Quá trình suy luận được thực hiện hoàn toàn trên điện thoại theo các bước sau:

1. Người dùng chụp ảnh bằng camera hoặc chọn ảnh có sẵn trong thư viện.

2. Ảnh được cắt theo tỷ lệ 1:1 để tập trung vùng lá, hoa hoặc quả cần chẩn đoán.
3. Ảnh được sửa hướng và resize về 224×224 pixel.
4. Ảnh JPEG được giải mã, chuyển từ RGBA sang RGB và tạo mảng `Float32Array`.
5. Giá trị pixel được giữ trong miền 0–255, phù hợp với preprocessing đã được nhúng trong model.
6. Tensor đầu vào được truyền cho TFLite và nhận về vector tám giá trị Softmax.
7. Kết quả được sắp xếp theo điểm từ cao xuống thấp và ánh xạ sang tên tiếng Việt.

Ứng dụng hiển thị lớp có điểm cao nhất và ba khả năng gần nhất. Khi điểm lớn nhất nhỏ hơn 0,6, giao diện cảnh báo người dùng rằng kết quả chưa chắc chắn và đề nghị chụp lại ảnh gần hơn, đủ sáng hơn. Ngưỡng này chỉ là quy tắc giao diện hỗ trợ người dùng, không phải ngưỡng xác suất đã được hiệu chuẩn thống kê. Mô hình hiện không có lớp `unknown`, vì vậy ảnh nằm ngoài tám nhóm đã học vẫn có thể bị gán vào một trong tám lớp.



Hình 4.4: Giao diện nhận diện bệnh bằng mô hình TensorFlow Lite trên điện thoại

4.2 Kết quả giao diện ứng dụng di động

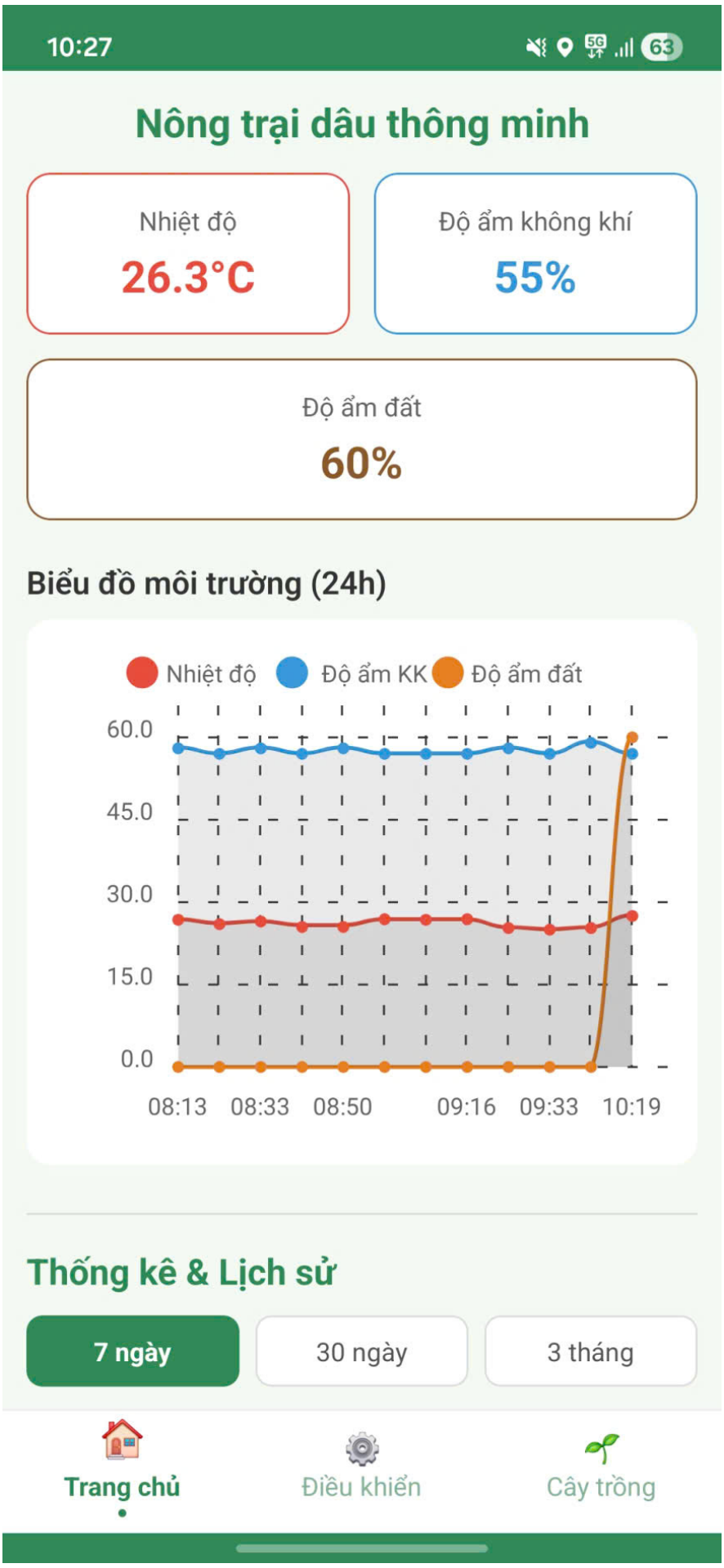
Ứng dụng được xây dựng bằng React Native và tổ chức thành ba tab chính là Trang chủ, Điều khiển và Cây trồng. Các màn hình sử dụng chung cơ chế xác định `device_id` để truy vấn dữ liệu đúng thiết bị. Supabase được sử dụng cho xác thực, lưu dữ liệu và đồng bộ Realtime, trong khi Cloudinary được sử dụng để lưu ảnh của cây trồng.

4.2.1 Màn hình Trang chủ

Màn hình Trang chủ hiển thị các giá trị mới nhất của nhiệt độ, độ ẩm không khí và độ ẩm đất. Khi mở màn hình, ứng dụng truy vấn bảng `sensor_readings` trong 24 giờ gần nhất và lấy mẫu đều số điểm để vẽ biểu đồ, giúp hạn chế tình trạng biểu đồ quá dày trên màn hình điện thoại.

Bên cạnh biểu đồ 24 giờ, ứng dụng cho phép người dùng lựa chọn khoảng 7 ngày, 30 ngày hoặc 3 tháng. Dữ liệu được hiển thị thành các biểu đồ riêng cho nhiệt độ, độ ẩm không khí và độ ẩm đất. Màn hình đồng thời lấy cảnh báo từ hai bảng `alertsic` và `alertsplant`, sau đó gộp và sắp xếp theo thời gian để hiển thị lịch sử cảnh báo mới nhất.

Kênh Realtime `sensor-readings-<deviceId>` lắng nghe sự kiện INSERT trên bảng `sensor_readings`. Khi ESP32 gửi một bản đo mới, giao diện cập nhật giá trị mới nhất và dữ liệu biểu đồ 24 giờ mà không cần người dùng tải lại trang.

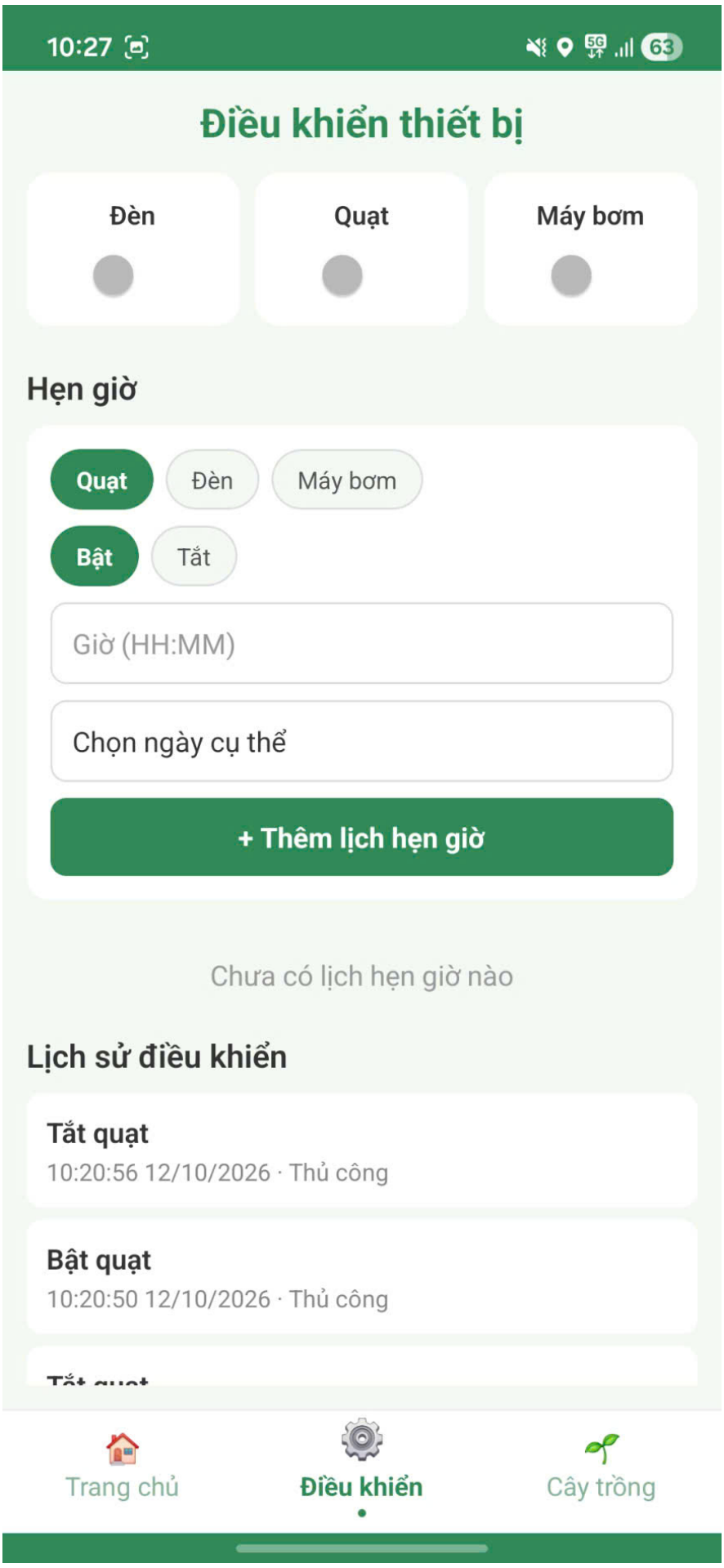


Hình 4.5: Giao diện Trang chủ hiển thị dữ liệu cảm biến và biểu đồ môi trường

4.2.2 Màn hình Điều khiển

Màn hình Điều khiển hiển thị trạng thái quạt, đèn và máy bơm ở lớp giao diện. Khi người dùng thay đổi công tắc, ứng dụng cập nhật bảng `actuator_states`, chuyển chế độ sang `manual` và ghi lịch sử thao tác vào bảng `control_logs`. Màn hình đăng ký Realtime cho cả bảng trạng thái và bảng log để đồng bộ giao diện khi dữ liệu thay đổi.

Ứng dụng cũng cho phép tạo lịch điều khiển bằng cách lựa chọn thiết bị, hành động bật hoặc tắt, thời gian và các ngày trong tuần. Các cấu hình lịch được lưu trong bảng `schedules`; người dùng có thể tạo, bật/tắt hoặc xóa lịch. Tuy nhiên, trong các tệp mã nguồn hiện tại chưa có tiến trình backend hoặc nhánh firmware đọc bảng `schedules` theo thời gian để tự thực thi. Vì vậy phần hẹn giờ hiện mới hoàn thiện ở mức quản lý cấu hình lịch.

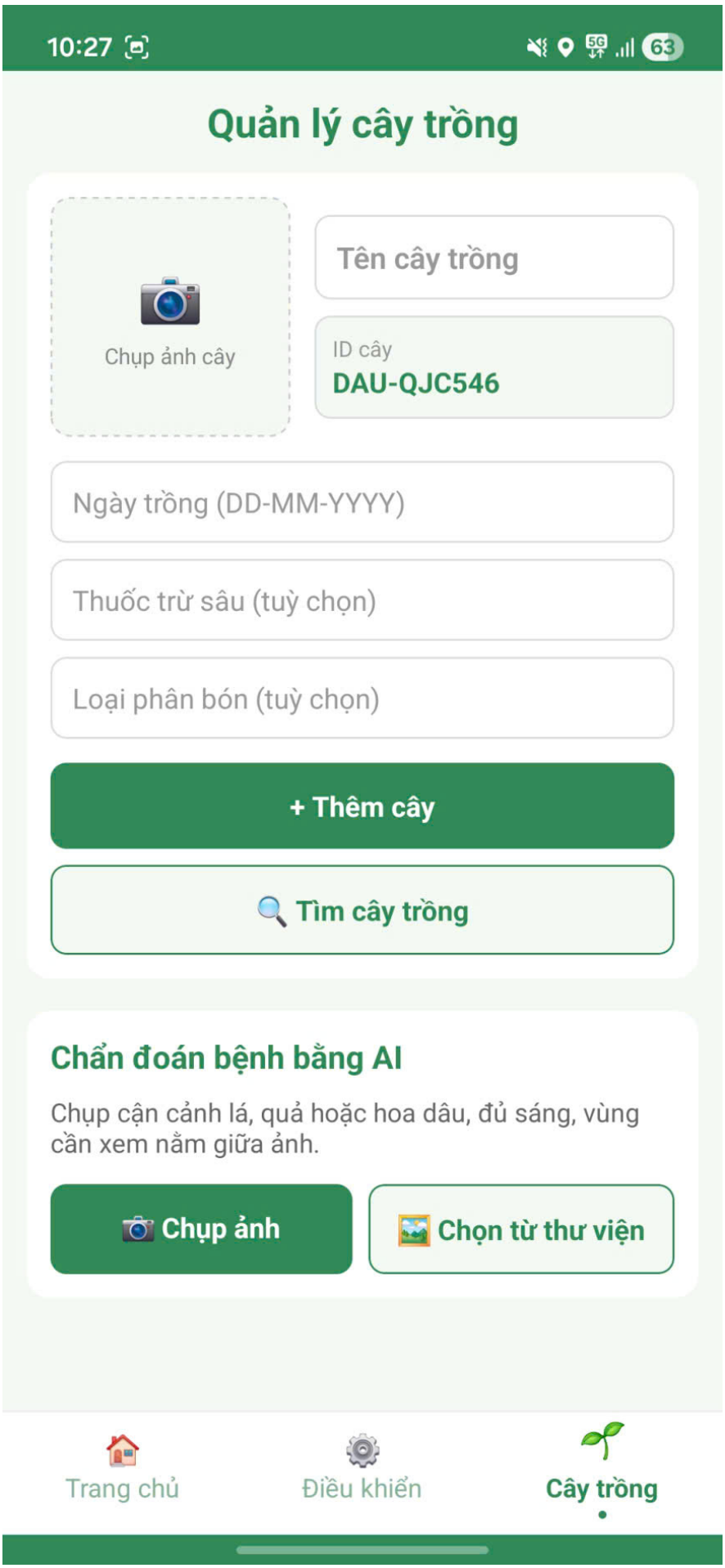


Hình 4.6: Giao diện điều khiển thiết bị và quản lý lịch hoạt động

4.2.3 Màn hình Cây trồng

Màn hình Cây trồng cho phép tạo hồ sơ cây với các thông tin gồm tên cây, mã cây sinh tự động, ngày trồng, thuốc trừ sâu và loại phân bón. Người dùng có thể chụp ảnh cây bằng camera. Ảnh được cắt theo tỷ lệ 1:1 và nén trước khi tải lên Cloudinary. Sau khi Cloudinary trả về URL, ứng dụng lưu URL vào bảng `plant_photos` để liên kết ảnh với bản ghi cây tương ứng trong Supabase.

Khi hiển thị danh sách, ứng dụng lấy ảnh mới nhất của mỗi cây làm thumbnail. Người dùng có thể tìm cây theo mã và lọc theo khoảng ngày trồng. Component `DiseaseScanner` cũng được đặt trực tiếp trong màn hình này, giúp chức năng quản lý cây và chẩn đoán bệnh xuất hiện trong cùng một khu vực chức năng.



Hình 4.7: Giao diện quản lý hồ sơ cây và hình ảnh cây trồng

4.3 Kết quả phần cứng và chương trình nhúng

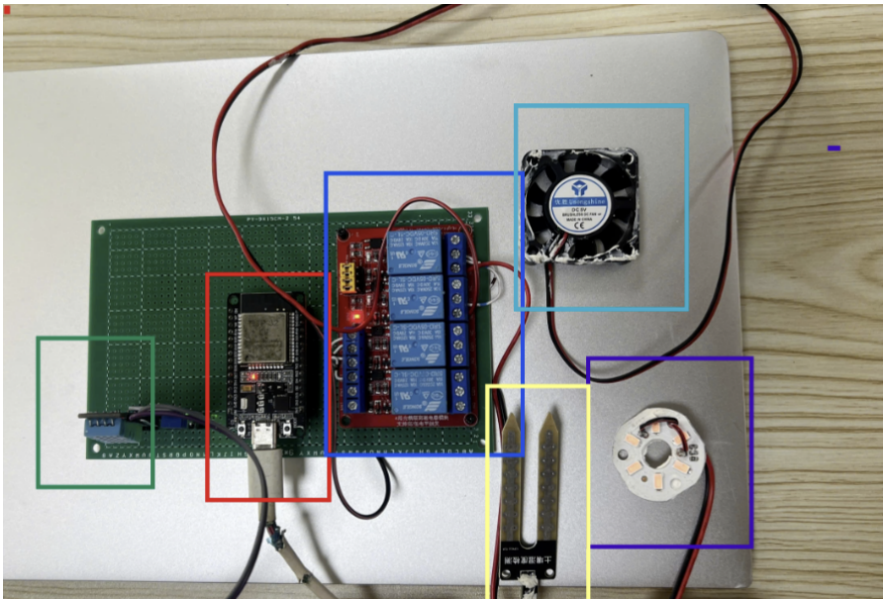
4.3.1 Kết nối phần cứng

ESP32 được sử dụng làm bộ điều khiển trung tâm. Cảm biến DHT11 kết nối với GPIO 4 để cung cấp nhiệt độ và độ ẩm không khí; cảm biến độ ẩm đất sử dụng ngõ vào analog GPIO 34. Hai relay được điều khiển bởi GPIO 26 và GPIO 27 tương ứng với quạt và đèn. GPIO 0 được dùng cho nút reset và GPIO 2 được dùng làm LED trạng thái. Cấu hình phần cứng được trình bày trong Bảng 4.2.

Bảng 4.2: Phân công chân phần cứng theo firmware hiện tại

Thành phần	**Chân ESP32**
DHT11	GPIO 4
Cảm biến độ ẩm đất	GPIO 34
Relay quạt	GPIO 26
Relay đèn	GPIO 27
Nút RESET	GPIO 0
LED trạng thái	GPIO 2

Trong firmware hiện tại, relay được cấu hình theo mức kích hoạt cao. Vì vậy khi thay đổi sang module relay kích mức thấp cần đảo logic điều khiển hoặc thay đổi tham số cấu hình tương ứng. Phần công suất của relay cũng cần được lựa chọn phù hợp với điện áp và dòng tải của quạt và đèn.



Hình 4.8: Mô hình phần cứng của hệ thống giám sát và điều khiển

4.3.2 Thu thập và xử lý dữ liệu cảm biến

Firmware đọc và gửi dữ liệu cảm biến theo chu kỳ 10 giây. Giá trị DHT11 được kiểm tra trước khi sử dụng; ngoài kiểm tra `NaN`, chương trình còn giới hạn nhiệt độ trong khoảng hợp lý 0–50 °C và độ ẩm không khí trong khoảng 20–90%. Chỉ sau lần đọc DHT hợp lệ đầu tiên, biến `sensorDataValid` mới cho phép chế độ tự động sử dụng dữ liệu cảm biến.

Cảm biến độ ẩm đất được đọc nhiều lần và lấy giá trị trung bình nhằm giảm nhiễu tức thời. Hai điểm hiệu chuẩn được đặt tại raw 4095 cho trạng thái khô và raw 2110 cho trạng thái ướt. Giá trị ADC sau đó được ánh xạ sang phần trăm từ 0 đến 100%. Các giá trị hiệu chuẩn này phù hợp với bộ cảm biến sử dụng trong mô hình thử nghiệm và cần được hiệu chuẩn lại khi thay đầu dò hoặc thay môi trường đất.

Nếu DHT11 đọc lỗi liên tiếp ba lần, firmware tạo cảnh báo lỗi cảm biến. Khi cảm biến hoạt động trở lại, hệ thống tạo sự kiện phục hồi. Cách xử lý này giúp phân biệt giữa một lần đọc lỗi ngẫu nhiên và lỗi kéo dài của phần cứng cảm biến.

4.3.3 Kết quả điều khiển thủ công và tự động

Ở chế độ thủ công, ESP32 kiểm tra trạng thái điều khiển trên Supabase theo chu kỳ 3 giây. Khi `mode=manual`, trạng thái quạt và đèn trong bảng `actuator_states` được sử dụng để đặt mức GPIO tương ứng. Sau khi trạng thái thay đổi, hệ thống đồng bộ kết quả về backend và ghi log điều khiển.

Ở chế độ tự động, quạt được điều khiển từ cả nhiệt độ và độ ẩm không khí. Quạt bật khi nhiệt độ lớn hơn 30 °C hoặc độ ẩm không khí lớn hơn 80%, và chỉ tắt khi nhiệt độ giảm dưới 28 °C và độ ẩm giảm dưới 75%. Việc dùng hai ngưỡng bật/tắt tạo vùng hysteresis, hạn chế relay đóng ngắt liên tục khi dữ liệu dao động gần ngưỡng.

Đối với relay đèn, firmware hiện sử dụng độ ẩm đất làm điều kiện tự động với ngưỡng 60%. Điều này phản ánh đúng logic đang có trong chương trình nhúng, tuy nhiên khi triển khai thực tế cần xác định rõ vai trò thiết bị chấp hành tương ứng với điều kiện độ ẩm đất. Nếu mục tiêu là tưới nước, hệ thống nên bổ sung relay máy bơm riêng thay vì dùng relay đèn.

Mặc dù ứng dụng đã có trường `pump_state` và giao diện máy bơm, firmware hiện chưa khai báo chân relay và nhánh điều khiển máy bơm. Vì vậy chức năng máy bơm chưa được xem là phần cứng đã hoàn thiện của phiên bản hiện tại.

4.3.4 Kết quả cơ chế cảnh báo và phục hồi

Firmware sử dụng cơ chế debounce và hysteresis cho các cảnh báo môi trường. Một trạng thái bất thường thông thường phải xuất hiện liên tiếp hai lần đọc trước khi được kích hoạt. Sau khi cảnh báo đã kích hoạt, trạng thái chỉ được xem là phục hồi khi giá trị đi qua ngưỡng phục hồi tương ứng. Nếu trạng thái bất thường kéo dài 15 phút hoặc đi vào vùng nghiêm trọng, mức cảnh báo có thể được nâng lên `critical`; cảnh báo đang tồn tại được nhắc lại theo chu kỳ 30 phút.

Các ngưỡng chính trong firmware được trình bày trong Bảng 4.3.

Bảng 4.3: Một số ngưỡng cảnh báo môi trường trong firmware

Đại lượng	**Bắt đầu cảnh báo**	**Phục hồi**	**Nghiêm trọng**
Độ ẩm đất thấp	$\leq 35\%$	$\geq 40\%$	$\leq 15\%$
Độ ẩm đất cao	$\geq 85\%$	$\leq 80\%$	$\geq 95\%$
Nhiệt độ cao	$\geq 35°C$	$\leq 32°C$	$\geq 40°C$
Nhiệt độ thấp	$\leq 10°C$	$\geq 13°C$	$\leq 2°C$
Độ ẩm không khí thấp	$\leq 40\%$	$\geq 45\%$	–
Độ ẩm không khí cao	$\geq 85\%$	$\leq 80\%$	–

Ngoài ra, firmware có watchdog phần cứng với timeout 60 giây và cơ chế xử lý mất Wi-Fi. Nếu thiết bị mất kết nối kéo dài khoảng 15 giây, chương trình chủ động khởi động lại để thử phục hồi kết nối. Nút reset hỗ trợ hai mức thao tác: xóa riêng cấu hình Wi-Fi khi giữ đủ thời gian thứ nhất và xóa toàn bộ cấu hình khi giữ lâu hơn.

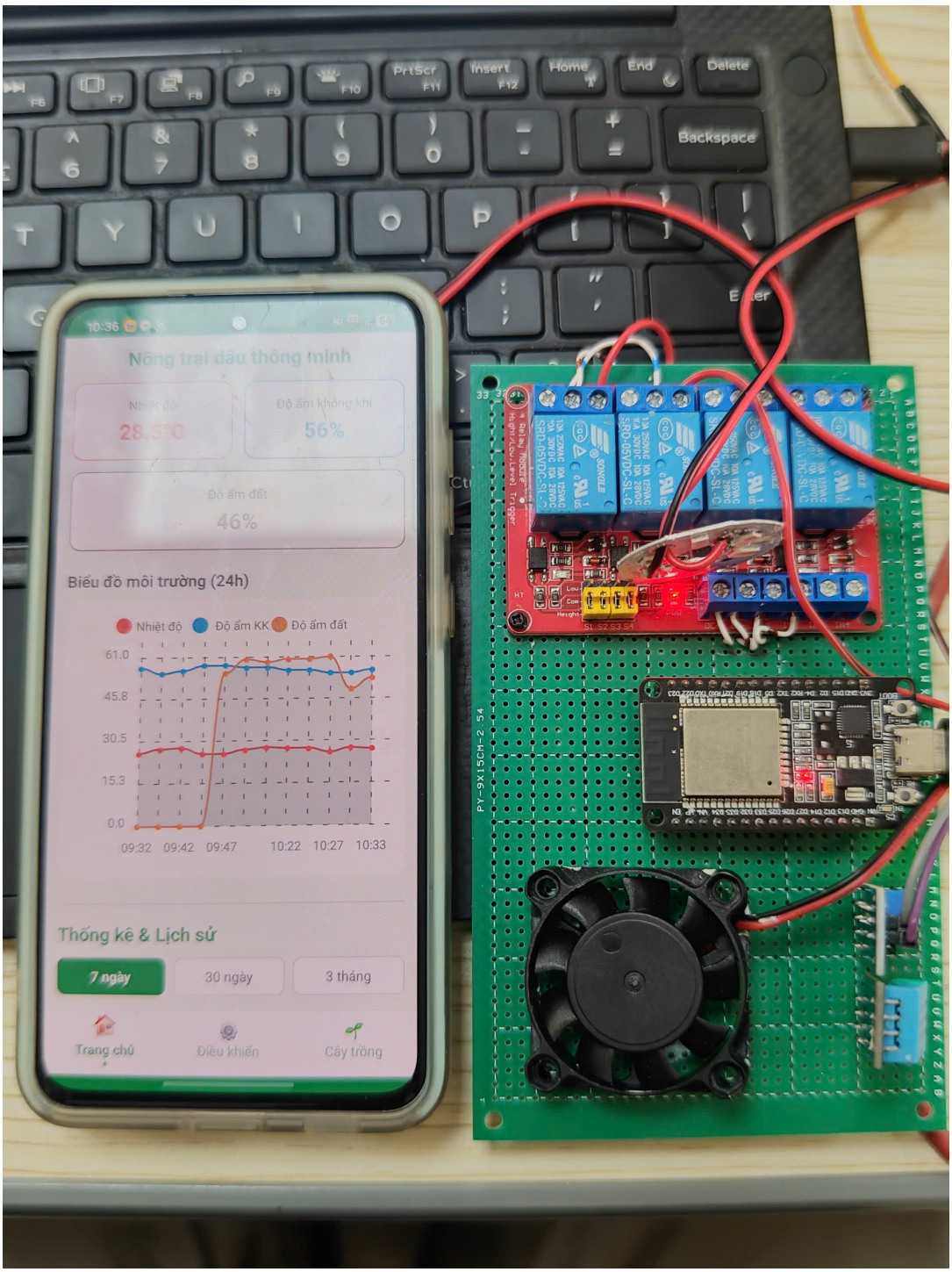
4.4 Kết quả tích hợp hệ thống

Sau khi các thành phần được kết nối, hệ thống hình thành hai luồng xử lý chính. Luồng IoT chịu trách nhiệm thu thập dữ liệu môi trường, đồng bộ trạng thái và điều khiển thiết bị; luồng AI chịu trách nhiệm phân tích ảnh bệnh trực tiếp trên điện thoại.

Đối với luồng IoT, ESP32 đọc cảm biến, chuyển đổi dữ liệu và gửi bản ghi lên bảng `sensor_readings`. Supabase lưu dữ liệu trong PostgreSQL và phát sự kiện Realtime. Ứng dụng React Native nhận sự kiện để cập nhật các thẻ thông số và biểu đồ. Theo chiều ngược lại, thao tác điều khiển của người dùng được ghi vào `actuator_states`; ESP32 định kỳ đọc trạng thái này và tác động lên relay khi ở chế độ thủ công.

Đối với quản lý cây, metadata được lưu trong bảng `plants`, còn ảnh được tải lên Cloudinary và URL được lưu trong `plant_photos`. Cách tổ chức này giúp ứng dụng quản lý được hồ sơ cây mà không cần lưu trực tiếp tệp ảnh lớn trong cơ sở dữ liệu quan hệ.

Đối với luồng AI, ảnh được xử lý hoàn toàn ở phía ứng dụng, tạo tensor đầu vào cho TFLite và nhận vector dự đoán tám lớp. Quá trình này không phụ thuộc vào một máy chủ AI riêng và có thể tiếp tục thực hiện suy luận sau khi model đã được tải trên thiết bị, ngay cả khi kết nối đến backend tạm thời không ổn định.



Hình 4.9: Kết quả tích hợp các thành phần của hệ thống

Mức độ hiện thực của các chức năng chính được tổng hợp trong Bảng 4.4.

Bảng 4.4: Mức độ hiện thực các chức năng trong phiên bản mã nguồn hiện tại

Nhóm chức năng	Nội dung	Trạng thái
Giám sát môi trường	Đọc nhiệt độ, độ ẩm không khí, độ ẩm đất và gửi lên Supabase	Đã hiện thực
Realtime	Cập nhật dữ liệu cảm biến mới trên ứng dụng	Đã hiện thực
Điều khiển quạt, đèn	Manual và Auto qua ESP32	Đã hiện thực
Cảnh báo	Cảnh báo môi trường, lỗi DHT11 và trạng thái phục hồi	Đã hiện thực
Quản lý cây	Thêm, tìm, lọc cây và lưu ảnh qua Cloudinary	Đã hiện thực
Nhận diện AI	TFLite chạy cục bộ, tám lớp, hiển thị Top 3	Đã hiện thực
Máy bơm	Có UI và trường dữ liệu nhưng chưa có relay trong firmware	Chưa hoàn thiện
Hẹn giờ	Tạo và quản lý bảng `schedules`	Chưa có bộ thực thi

4.5 Đánh giá và nhận xét

Kết quả xây dựng cho thấy kiến trúc đề xuất có thể kết hợp phần cứng IoT, backend, ứng dụng di động và mô hình học sâu trong một hệ thống thống nhất. ESP32 đảm nhiệm thu thập và điều khiển tại hiện trường; Supabase đóng vai trò trung gian lưu trữ và đồng bộ dữ liệu; React Native cung cấp giao diện quản lý; TensorFlow Lite bổ sung khả năng nhận diện bệnh trên thiết bị di động.

Ở phần IoT, việc tách luồng dữ liệu cảm biến và luồng điều khiển qua cơ sở dữ liệu giúp điện thoại không cần kết nối trực tiếp đến ESP32 hoặc cùng mạng nội bộ. Chu kỳ gửi dữ liệu 10 giây và chu kỳ kiểm tra điều khiển 3 giây phù hợp với mô hình giám sát thử nghiệm, đồng thời cơ chế Realtime giúp giao diện phản ánh bản ghi mới mà không cần truy vấn liên tục.

Ở phần chương trình nhúng, việc bổ sung kiểm tra dữ liệu hợp lệ, trung bình nhiều mẫu ADC, hysteresis, debounce cảnh báo, watchdog và xử lý mất Wi-Fi giúp firmware ổn định hơn so với cách xử lý trực tiếp từng mẫu đo. Tuy nhiên, các ngưỡng cảnh báo và ngưỡng điều khiển hiện vẫn là giá trị cấu hình cố định, chưa được cá nhân hóa theo giống cây, giai đoạn sinh trưởng hoặc điều kiện thực tế của từng nông trại.

Ở phần ứng dụng, ba màn hình chính đáp ứng các yêu cầu giám sát, điều khiển và quản lý cây. Việc tích hợp `DiseaseScanner` vào màn hình Cây trồng giúp đưa chức năng AI vào luồng sử dụng thực tế. Tuy vậy, chức năng hẹn giờ mới dừng ở việc lưu cấu hình và chức năng máy bơm mới dừng ở lớp giao diện, do đó chưa thể xem hai chức năng này là đã hoàn thiện toàn hệ thống.

Ở phần AI, quá trình huấn luyện đã tạo được model Keras và hai biến thể TFLite, đồng thời model FP32 đã được nhúng và gọi trực tiếp từ ứng dụng. Đây là kết quả quan trọng vì

chứng minh được đường triển khai từ notebook huấn luyện đến thiết bị di động. Mặt khác, mô hình chưa có lớp unknown, điểm Softmax chưa được hiệu chuẩn và cách chia dữ liệu chưa bảo đảm group-aware split. Do đó kết quả nhận diện cần được xem là công cụ hỗ trợ, không thay thế hoàn toàn đánh giá của người có chuyên môn.

Từ các kết quả trên có thể nhận thấy mô hình thử nghiệm đã hoàn thành phần lớn các mục tiêu cốt lõi của đề tài: thu thập dữ liệu môi trường, lưu trữ và đồng bộ dữ liệu, điều khiển quạt và đèn, quản lý hồ sơ cây và thực hiện nhận diện bệnh trực tiếp trên điện thoại. Những phần chưa hoàn thiện được xác định rõ để tiếp tục phát triển ở Chương 5.

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Đề tài "Phát triển ứng dụng di động quản lý và giám sát nông trại dâu tích hợp IoT và AI" được thực hiện với mục tiêu xây dựng một hệ thống có khả năng theo dõi điều kiện môi trường, hỗ trợ điều khiển thiết bị và nhận diện một số bệnh phổ biến trên cây dâu tây. Sau quá trình nghiên cứu, thiết kế và triển khai, nhóm đã xây dựng được mô hình hệ thống gồm phần cứng ESP32, cảm biến môi trường, cơ sở dữ liệu Supabase, ứng dụng di động React Native, dịch vụ lưu ảnh Cloudinary và mô hình MobileNetV3-Large triển khai bằng TensorFlow Lite.

Ở khối phần cứng, ESP32 đã thực hiện được việc đọc nhiệt độ, độ ẩm không khí và độ ẩm đất. Dữ liệu cảm biến được xử lý, kiểm tra tính hợp lệ và gửi lên Supabase theo chu kỳ. Firmware đồng thời điều khiển quạt và đèn thông qua relay, hỗ trợ cả chế độ thủ công và tự động. Các cơ chế hysteresis, debounce cảnh báo, phát hiện lỗi cảm biến, watchdog và xử lý mất Wi-Fi được bổ sung nhằm tăng tính ổn định của thiết bị trong quá trình vận hành.

Ở khối backend, Supabase được sử dụng làm nền tảng trung gian giữa ESP32 và ứng dụng. Dữ liệu môi trường, trạng thái thiết bị, cảnh báo, lịch sử điều khiển, lịch hẹn và hồ sơ cây được tổ chức thành các nhóm dữ liệu riêng. Supabase Realtime giúp ứng dụng nhận thay đổi mới của cơ sở dữ liệu mà không cần liên tục gửi yêu cầu truy vấn. Cách tổ chức này cho phép thiết bị phần cứng và điện thoại trao đổi dữ liệu qua Internet mà không cần nằm trong cùng một mạng nội bộ.

Ở khối ứng dụng, React Native đã cung cấp ba nhóm chức năng chính gồm Trang chủ, Điều khiển và Cây trồng. Trang chủ hiển thị giá trị cảm biến mới nhất, biểu đồ 24 giờ, thống kê theo các khoảng thời gian và lịch sử cảnh báo. Màn hình Điều khiển cho phép bật hoặc tắt thiết bị, theo dõi log điều khiển và tạo cấu hình lịch. Màn hình Cây trồng hỗ trợ thêm cây, lưu thông tin canh tác, chụp ảnh, tải ảnh lên Cloudinary, tìm cây theo mã và lọc theo ngày trồng.

Đối với bài toán AI, nhóm sử dụng MobileNetV3-Large với phương pháp học chuyển giao để phân loại tám nhóm gồm bảy bệnh và một lớp khỏe mạnh. Bộ dữ liệu sau quá trình kiểm tra và loại ảnh trùng còn 2915 ảnh, được chia thành train, validation và test. Mô hình được huấn luyện theo hai giai đoạn gồm huấn luyện đầu phân loại và fine-tuning một phần backbone. Sau huấn luyện, model được xuất sang TensorFlow Lite ở hai dạng FP32 và FP16.

Phiên bản ứng dụng mới nhất đã tích hợp trực tiếp model TensorFlow Lite thông qua `react-native-fast-tflite`. Người dùng có thể chụp ảnh hoặc chọn ảnh từ thư viện; ứng dụng resize ảnh về 224×224, tạo tensor RGB kiểu `float32` và chạy suy luận ngay trên điện thoại. Kết quả được ánh xạ sang tên tiếng Việt và hiển thị cùng ba khả năng có điểm cao nhất. Việc suy luận cục bộ giúp giảm phụ thuộc vào máy chủ AI và phù hợp với định hướng

triển khai trên thiết bị di động của đề tài.

Từ các kết quả đạt được, có thể kết luận rằng hướng kết hợp IoT và AI trong cùng một ứng dụng quản lý nông trại là khả thi đối với mô hình thử nghiệm. Hệ thống hiện tại đã hình thành được chuỗi xử lý tương đối đầy đủ từ thu thập dữ liệu tại hiện trường, truyền và lưu trữ dữ liệu, hiển thị trên ứng dụng, gửi lệnh điều khiển đến thiết bị, quản lý cây và phân tích ảnh bệnh. Đây là nền tảng để tiếp tục cải tiến hệ thống theo hướng ổn định hơn, chính xác hơn và phù hợp hơn với điều kiện canh tác thực tế.

5.2 Hạn chế của đề tài

Bên cạnh các kết quả đã đạt được, hệ thống vẫn còn một số hạn chế cần được xem xét khi triển khai ở quy mô thực tế.

Trước hết, cảm biến DHT11 có độ chính xác và tốc độ phản hồi còn hạn chế so với các cảm biến môi trường chuyên dụng. Cảm biến độ ẩm đất analog cũng chịu ảnh hưởng của loại đầu dò, vị trí đặt cảm biến, tính dẫn điện của đất và điều kiện sử dụng lâu dài. Hai giá trị hiệu chuẩn ADC hiện được xác định cho bộ cảm biến thử nghiệm, vì vậy cần hiệu chuẩn lại khi thay đầu dò hoặc thay môi trường đất.

Các ngưỡng điều khiển và cảnh báo trong firmware đang được cấu hình cố định. Trong thực tế, ngưỡng phù hợp có thể thay đổi theo giống dâu, giai đoạn sinh trưởng, mùa vụ, vị trí trồng và điều kiện nhà kính. Vì vậy việc sử dụng một bộ ngưỡng duy nhất chưa phản ánh đầy đủ sự biến đổi của môi trường canh tác.

Phần cứng hiện tại mới có relay thực tế cho quạt và đèn. Ứng dụng đã có trường `pump_state` và giao diện máy bơm nhưng firmware chưa khai báo GPIO, relay và logic điều khiển tương ứng. Do đó chức năng máy bơm chưa hoạt động hoàn chỉnh từ ứng dụng đến phần cứng.

Chức năng hẹn giờ hiện cho phép người dùng tạo, bật/tắt và xóa các bản ghi trong bảng `schedules`. Tuy nhiên, hệ thống chưa có tiến trình backend, cron job hoặc nhánh firmware chịu trách nhiệm đọc lịch đúng thời điểm và tự động thực hiện lệnh. Vì vậy giao diện hẹn giờ hiện mới đóng vai trò quản lý cấu hình.

Ứng dụng và firmware vẫn phụ thuộc vào kết nối Internet đối với các chức năng trao đổi dữ liệu qua Supabase và lưu ảnh trên Cloudinary. Firmware đã có cơ chế thử phục hồi khi mất Wi-Fi, nhưng ứng dụng chưa có cơ chế lưu offline và đồng bộ lại dữ liệu đầy đủ khi kết nối mạng trở lại.

Về dữ liệu AI, số lượng ảnh giữa các lớp còn mất cân bằng. Lớp ít mẫu nhất chỉ có 97 ảnh, trong khi lớp nhiều mẫu nhất có 574 ảnh. Quá trình huấn luyện đã sử dụng `class_weight`, tuy nhiên sự chênh lệch số lượng ảnh vẫn có thể ảnh hưởng đến khả năng tổng quát hóa của các lớp có ít dữ liệu.

Phương pháp chia dữ liệu hiện mới loại ảnh trùng hoàn toàn bằng pixel hash và sau đó chia ngẫu nhiên theo tỷ lệ 70/15/15. Notebook chưa sử dụng `group_id` theo cây hoặc theo phiên chụp. Vì vậy vẫn có khả năng ảnh gần giống nhau của cùng một cây xuất hiện ở nhiều

tập khác nhau, làm cho kết quả trên tập test có thể lạc quan hơn so với khi đánh giá trên một nông trại hoàn toàn mới.

Mô hình AI hiện chỉ gồm tám lớp đã biết và không có lớp `unknown`. Do lớp cuối sử dụng Softmax, mô hình luôn đưa ra một lớp có điểm lớn nhất ngay cả khi ảnh đầu vào không phải dâu tây hoặc thuộc một bệnh chưa được huấn luyện. Ứng dụng có sử dụng ngưỡng 0,6 để cảnh báo kết quả thấp, nhưng ngưỡng này chưa được xây dựng từ một quy trình hiệu chuẩn xác suất hoặc đánh giá dữ liệu ngoài phân phối.

Mặc dù model TensorFlow Lite đã được tích hợp vào ứng dụng, các tệp hiện có chưa thể hiện một báo cáo benchmark đầy đủ trên nhiều thiết bị Android về thời gian suy luận, mức sử dụng bộ nhớ, mức tiêu thụ năng lượng và chênh lệch kết quả giữa Keras, TFLite FP32 và TFLite FP16. Đây là phần cần bổ sung trước khi đánh giá sâu về hiệu quả triển khai trên thiết bị di động.

Cuối cùng, vấn đề bảo mật cần được chú ý khi triển khai thực tế. Các khóa truy cập dịch vụ không nên được nhúng với quyền cao trực tiếp trong firmware hoặc mã ứng dụng. Hệ thống cần sử dụng Row Level Security, nguyên tắc quyền tối thiểu, cơ chế quản lý secret phù hợp và lớp API trung gian cho các thao tác nhạy cảm.

5.3 Hướng phát triển

Từ các hạn chế trên, hệ thống có thể được tiếp tục phát triển theo các hướng sau:

- Bổ sung relay và logic firmware cho máy bơm để hoàn thiện chức năng tưới tự động. Khi đó độ ẩm đất có thể được sử dụng để điều khiển trực tiếp máy bơm thay vì chỉ tạo cảnh báo hoặc tác động lên một thiết bị không liên quan đến tưới.

- Thay DHT11 bằng cảm biến có độ chính xác cao hơn như DHT22, SHT30 hoặc SHT31. Đối với độ ẩm đất, có thể sử dụng cảm biến điện dung để giảm ảnh hưởng của quá trình ăn mòn điện cực khi vận hành lâu dài.

- Bổ sung các cảm biến ánh sáng, lượng mưa, pH, EC hoặc các thông số dinh dưỡng. Nguồn dữ liệu đa dạng hơn sẽ giúp hệ thống đánh giá điều kiện sinh trưởng của cây đầy đủ hơn và tạo cơ sở cho các thuật toán điều khiển nâng cao.

- Cho phép cấu hình ngưỡng cảnh báo và ngưỡng điều khiển từ ứng dụng hoặc backend thay vì cố định trong firmware. Các bộ ngưỡng có thể được lưu theo giống cây, giai đoạn sinh trưởng hoặc khu vực trồng.

- Hoàn thiện bộ thực thi lịch bằng Supabase Edge Functions, cron job hoặc cơ chế lập lịch trực tiếp trên ESP32. Khi đó các bản ghi trong bảng `schedules` có thể thực sự kích hoạt lệnh theo thời gian đã cấu hình.

- Xây dựng cơ chế lưu cục bộ và đồng bộ lại dữ liệu khi mất mạng. ESP32 có thể lưu tạm bản đo chưa gửi được, trong khi ứng dụng có thể cache các dữ liệu cần thiết để tăng khả năng sử dụng khi kết nối Internet không ổn định.

- Thu thập thêm ảnh thực tế trong điều kiện nông trại với nhiều mức ánh sáng, góc chụp, thiết bị camera, giống dâu và giai đoạn bệnh khác nhau. Dữ liệu mới cần được quản lý theo mã cây hoặc mã phiên chụp để có thể thực hiện group-aware split.
- Mở rộng bài toán AI bằng lớp `unknown` hoặc các phương pháp phát hiện dữ liệu ngoài phân phối. Hướng này giúp hạn chế trường hợp model đưa ra dự đoán có vẻ chắc chắn cho ảnh không thuộc phạm vi tám lớp đã huấn luyện.
- Thực hiện calibration xác suất và lựa chọn ngưỡng cảnh báo dựa trên dữ liệu validation thay vì dùng ngưỡng cố định theo kinh nghiệm. Có thể sử dụng các phương pháp như temperature scaling để làm điểm dự đoán phản ánh mức độ tin cậy tốt hơn.
- Benchmark và tối ưu các phiên bản TensorFlow Lite FP32, FP16 và INT8 trên nhiều điện thoại Android. Mục tiêu là giảm kích thước model và thời gian suy luận mà vẫn duy trì độ chính xác chấp nhận được.
- Nghiên cứu chuyển từ bài toán phân loại toàn ảnh sang phát hiện đối tượng hoặc phân đoạn vùng bệnh. Khi đó ứng dụng không chỉ trả về tên bệnh mà còn có thể đánh dấu khu vực bất thường trên lá, hoa hoặc quả để người dùng dễ kiểm chứng kết quả.
- Lưu lịch sử chẩn đoán AI cho từng cây. Mỗi lần nhận diện có thể lưu ảnh, lớp dự đoán, điểm model, thời gian chẩn đoán và ghi chú xử lý, từ đó hình thành hồ sơ theo dõi diễn biến bệnh theo thời gian.
- Mở rộng kiến trúc theo `device_id` để quản lý nhiều ESP32, nhiều luống dâu hoặc nhiều khu vực trong cùng một nông trại. Ứng dụng có thể bổ sung màn hình chọn thiết bị và tổng hợp trạng thái của nhiều node cảm biến.
- Tăng cường bảo mật bằng Row Level Security, quyền truy cập tối thiểu, quản lý khóa an toàn và lớp API trung gian. Đối với các chức năng điều khiển thiết bị, có thể bổ sung xác thực vai trò người dùng và ghi log đầy đủ để phục vụ truy vết.

Những hướng phát triển trên tập trung vào ba mục tiêu chính: tăng độ tin cậy của dữ liệu IoT, hoàn thiện chức năng điều khiển tự động và nâng cao khả năng tổng quát hóa của mô hình AI. Nếu được tiếp tục thử nghiệm với dữ liệu thực tế và thiết bị phần cứng hoàn chỉnh, hệ thống có thể phát triển từ mô hình đồ án thành một nền tảng hỗ trợ giám sát và quản lý nông trại dâu ở quy mô lớn hơn.

TÀI LIỆU THAM KHẢO

[1] M. Ayaz, M. Ammad-Uddin, Z. Sharif, A. Mansour, and E.-H. M. Aggoune, "Internet-of-things (IoT)-based smart agriculture: Toward making the fields talk," *IEEE Access*, vol. 7, pp. 129551–129583, 2019.

[2] Supabase, "Supabase Documentation." [Online]. Available: https://supabase.com/docs. Accessed: Oct. 27, 2026.

[3] Meta Platforms, Inc., "React Native Documentation." [Online]. Available: https://reactnative.dev/docs/getting-started. Accessed: Oct. 27, 2026.

[4] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, 2015.

[5] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010.

[6] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, "Searching for MobileNetV3," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 1314–1324, 2019.

MÃ NGUỒN ĐỀ TÀI

Toàn bộ mã nguồn của đề tài *Phát triển ứng dụng di động quản lý và giám sát nông trại dâu tích hợp IoT và AI* được lưu trữ trên nền tảng GitHub.

Mã nguồn bao gồm chương trình điều khiển ESP32, ứng dụng di động React Native và mô hình học sâu MobileNetV3-Large phục vụ nhận diện bệnh cây dâu tây.

Clip demo sản phẩm

Đường dẫn mã nguồn:

- **GitHub:** `https://github.com/timmie182/Strawberry_Agriculture_App`
- **Youtube-Clip demo:** `https://www.youtube.com/playlist?list=PLWY9NlAltIqg&si=1EQnq0oKwDd4dq7o`